

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

Работа допущена к защите

Директор ВШ КТиИС

_____ В.А. Сушников

«__» _____ 2025 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

РАБОТА БАКАЛАВРА

РАЗРАБОТКА МОБИЛЬНОГО ПРИЛОЖЕНИЯ ДЛЯ ПЕРЕВОДА ЯЗЫКА ЖЕСТОВ

по направлению подготовки

09.03.01 Информатика и вычислительная техника

код и наименование

направленность (профиль)

09.03.01_02 Технологии разработки программного обеспечения

код и наименование

Выполнил

студент гр.5130901/10203

<подпись>

П.В. Макарова

Руководитель

должность,

ученая степень, ученое звание

<подпись>

Н.В. Богач

Консультант

по нормоконтролю

<подпись>

А.Г. Новопашенный

Санкт-Петербург

2025

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

УТВЕРЖДАЮ

Директор ВШ КТиИС

В.А. Сушников

«__» _____ 2025 г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы

студенту Макаровой Полине Владиславовне, группа 5130901/10203

1. Тема работы: Разработка мобильного приложения для перевода языка жестов
2. Срок сдачи студентом законченной работы: 21.05.2025
3. Исходные данные по работе:
 - 3.1. Методические рекомендации по подготовке выпускных квалификационных работ
 - 3.2. Научные статьи на тему распознавания жестов
 - 3.3. Документации языков программирования Python и Kotlin, документация фреймворков Flask и Jetpack Compose
4. Содержание работы (перечень подлежащих разработке вопросов):
 - 4.1 Обзор существующих методов распознавания жестов
 - 4.2 Проектирование и реализация пользовательского интерфейса
 - 4.3 Разработка архитектуры системы распознавания жестов для мобильного приложения
 - 4.4 Реализация клиент-серверного приложения
5. Перечень графического материала (с указанием обязательных чертежей): -
6. Перечень используемых информационных технологий, в том числе программное обеспечение, облачные сервисы, базы данных и прочие сквозные цифровые технологии (при наличии):
 - 6.1 Языки программирования Python, Kotlin
 - 6.2 Библиотеки машинного обучения: TensorFlow, Keras
 - 6.3 Фреймворки Jetpack Compose, Flask
7. Консультанты по работе: консультант по нормоконтролю А.Г. Новопашенный
8. Дата выдачи задания 21.04.2025

Руководитель ВКР

(подпись)

Н.В. Богач

Задание принял к исполнению 21.04.2025

Студент

(подпись)

П.В. Макарова

РЕФЕРАТ

На 59 с., 14 рисунков, 4 таблицы, 8 приложений

КЛЮЧЕВЫЕ СЛОВА: РУССКИЙ ЖЕСТОВЫЙ ЯЗЫК, РАСПОЗНАВАНИЕ ЖЕСТОВ, МОБИЛЬНОЕ ПРИЛОЖЕНИЕ, НЕЙРОННАЯ СЕТЬ, MEDIAPIPE, КОМПЬЮТЕРНОЕ ЗРЕНИЕ, ANDROID.

Тема выпускной квалификационной работы: «Разработка мобильного приложения для перевода языка жестов».

Русский жестовый язык является основным средством общения для более чем 240 тысяч человек в России. Однако коммуникационный барьер между слышащими и неслышащими людьми остается серьезной социальной проблемой. Существующие решения для перевода жестового языка преимущественно реализованы в виде настольных приложений, что ограничивает их использование в повседневной жизни. Данная работа посвящена созданию мобильного приложения для распознавания жестов русского жестового языка в режиме реального времени.

Задачи, которые решались в ходе разработки:

- 1) Анализ особенностей русского жестового языка и существующих решений для его распознавания;
- 2) Проектирование клиент-серверной архитектуры системы распознавания жестов;
- 3) Разработка мобильного приложения с функциями захвата видео, детекции ключевых точек рук и визуализации результатов;
- 4) Реализация серверной части для обработки данных и распознавания жестов с помощью нейронных сетей;
- 5) Обучение и сравнение различных архитектур нейронных сетей (LSTM, GRU, 3D CNN) для выбора оптимального решения;
- 6) Тестирование системы на реальных устройствах и оценка производительности.

В результате работы создано полнофункциональное мобильное приложение, способное распознавать 30 жестов РЖЯ с точностью 98.53% при использовании LSTM-модели.

ABSTRACT

59 pages, 14 figures, 4 tables, 8 appendices

KEY WORDS: RUSSIAN SIGN LANGUAGE, GESTURE RECOGNITION, MOBILE APPLICATION, NEURAL NETWORK, MEDIAPIPE, COMPUTER VISION, ANDROID.

The subject of the graduate qualification work is «Development of a mobile application for sign language translation».

Russian Sign Language is the primary means of communication for more than 240,000 people in Russia. However, the communication barrier between hearing and deaf people remains a serious social problem. Existing solutions for sign language translation are predominantly implemented as desktop applications, which limit their use in everyday life. This work is devoted to creating a mobile application for recognizing Russian Sign Language gestures in real time.

Tasks solved during development:

- 1) Analysis of Russian Sign Language features and existing solutions for its recognition.
- 2) Design of client-server architecture for gesture recognition system.
- 3) Development of a mobile application with video capture, hand key point detection and visualization functions.
- 4) Implementation of server-side for data processing and gesture recognition using neural networks.
- 5) Training and comparison of different neural network architectures (LSTM, GRU, 3D CNN) to select the optimal solution.
- 6) Testing the system on real devices and performance evaluation.

As a result of the work, a fully functional mobile application was created, capable of recognizing 30 RSL gestures with 98.53% accuracy using the LSTM model.

СОДЕРЖАНИЕ

СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ.....	7
ВВЕДЕНИЕ.....	9
ГЛАВА 1. РУССКИЙ ЖЕСТОВЫЙ ЯЗЫК: СТРУКТУРА И ТЕХНОЛОГИИ. 11	
1.1. Обзор русского жестового языка.....	11
1.2. Анализ существующих решений	12
ГЛАВА 2. ВЫБОР СРЕДСТВ РЕАЛИЗАЦИИ И ПОСТАНОВКА ЗАДАЧИ.....	15
2.1. Анализ существующих решений	15
2.1.1. Языки программирования	15
2.1.2. Инструменты для обработки изображений и детекции рук	16
2.1.3. Нейронные сети для обучения модели	18
2.1.4. Набор данных для обучения нейросетевой модели	19
2.2. Постановка задачи и требований.....	20
2.2.1. Функциональные требования.....	20
2.2.2. Нефункциональные требования.....	21
ГЛАВА 3. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ СИСТЕМЫ.....	23
3.1. Выбор архитектуры системы.....	23
3.2. Проектирование клиента.....	24
3.2.1. Проектирование пользовательского интерфейса	24
3.2.2. Проектирование архитектуры клиента.....	25
3.3. Проектирование сервера.....	28
3.4. Проектирование базы данных	31
ГЛАВА 4. РЕАЛИЗАЦИЯ СИСТЕМЫ РАСПОЗНАВАНИЯ ЖЕСТОВ.....	33
4.1. Разработка клиентской части	33
4.1.1. Структура проекта.....	33
4.1.2. Захват и обработка видеопотока с камеры	34
4.1.3. Детекция ключевых точек рук с Mediapipe	34
4.1.4. Подготовка данных для передачи на сервер.....	35

4.1.5. Визуализация ключевых точек рук.....	36
4.1.6. Отображение результатов распознавания.....	38
4.2. Разработка сервера.....	40
4.2.1. Прием и валидация данных от клиента.....	40
4.2.2. Подготовка данных для нейронной сети	41
4.2.3. Процесс распознавания с помощью нейронной сети	41
4.3. Реализация и обучение нейронных сетей	42
ГЛАВА 5. ТЕСТИРОВАНИЕ МОБИЛЬНОГО ПРИЛОЖЕНИЯ	49
5.1. Тестирование системы	49
5.2. Совместимость с устройствами	51
5.3. Сравнение нейронных сетей	51
ЗАКЛЮЧЕНИЕ	55
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	57
Приложение 1. CameraHelper.....	60
Приложение 2. HandLandmarkHelper	64
Приложение 3. extractFeaturesFromLandmarks.....	69
Приложение 4. GesturePhaseDetector	71
Приложение 5. OverlayView.....	75
Приложение 6. SignLanguageApiService	78
Приложение 7. process_gesture	82
Приложение 8. recognize_gesture.....	84

СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ

ЖЯ	Жестовый язык
КЖЯ	Калькирующая жестовая речь
РЖЯ	Русский жестовый язык
3D CNN	3-Dimensional Convolutional Neural Network (трехмерная свёрточная нейронная сеть)
API	Application Programming Interface (программный интерфейс приложения)
APK	Android Package Kit (пакет приложения Android)
ASL	American Sign Language (американский жестовый язык)
CNN	Convolutional Neural Network (сверточная нейронная сеть)
CPU	Central Processing Unit (центральный процессор)
ER	Entity-Relationship (сущность-связь)
FPS	Frames Per Second (кадров в секунду)
GPU	Graphics Processing Unit (графический процессор)
GRU	Gated Recurrent Unit (управляемый рекуррентный блок)
HTTP	HyperText Transfer Protocol (протокол передачи гипертекста)
IDE	Integrated Development Environment (интегрированная среда разработки)
IP	Internet Protocol (интернет-протокол)
JSON	JavaScript Object Notation (текстовый формат обмена данными)
LSTM	Long Short-Term Memory (долгая краткосрочная память)
MVVM	Model-View-ViewModel (архитектурный паттерн)
ReLU	Rectified Linear Unit (выпрямленная линейная функция активации)

REST	Representational State Transfer (архитектурный стиль взаимодействия)
RGB	Red Green Blue (красный зеленый синий - цветовая модель)
SDK	Software Development Kit (комплект средств разработки)
SQL	Structured Query Language (структурированный язык запросов)
UI	User Interface (пользовательский интерфейс)
UX	User Experience (пользовательский опыт)
XML	eXtensible Markup Language (расширяемый язык разметки)
YUV	Цветовая модель с разделением яркости и цветности

ВВЕДЕНИЕ

Слух – один из основных органов чувств, благодаря которому человек способен коммуницировать с обществом, реагировать на опасность и ориентироваться в пространстве. Однако с каждым годом всё больше людей сталкиваются с проблемами восприятия звуков – как врождёнными, так и приобретёнными. По данным Всемирной организации здравоохранения более 430 миллионов человек в мире обладают серьезными нарушениями слуха [2], из них более 70 миллионов являются полностью глухими. В России, согласно прогнозам, к 2025 году число слабослышащих достигнет 19,5 миллионов человек [4].

Обеспечить эффективную коммуникацию такого количества людей помогают жестовые языки (ЖЯ) – визуально-пространственные лингвистические системы, позволяющие общаться посредством движений рук, мимики и артикуляции. Современная лингвистика официально признает такие системы полноценными естественными языками со своей грамматикой, синтаксисом и лексикой [3].

Всего в мире насчитывается свыше 300 различных национальных и региональных жестовых языков [8]. На межнациональном уровне, в частности на конференциях и иных мероприятиях, представители сообщества глухих общаются с помощью международной жестовой речи (International Sign, IS) [5]. В Российской Федерации, а также на территориях Белоруссии, Украины и ряда стран Средней Азии слабослышащие активно используют русский жестовый язык (РЖЯ), который получил официальное признание 30 декабря 2012 года [10]. По данным Всероссийской переписи населения 2020 года, РЖЯ является основным средством общения примерно для 240 тысяч человек [7], однако реальное число пользователей может быть значительно больше.

Жестовые языки существенно облегчают коммуникацию между людьми с нарушениями слуха, но барьер между глухими и слышащими сохраняется до сих пор. Отсутствие универсальных средств общения часто ограничивает доступ слабослышащих людей к образованию, трудоустройству и полноценному

участию в общественной жизни, что может привести к их социальной изоляции. Чтобы этого избежать, необходимо придумать решение, с помощью которого люди с разными возможностями слуха могли бы поддерживать взаимодействие в любое время.

В эру цифровых технологий особую актуальность приобретают автоматизированные системы перевода жестового языка. Большинство подобных программ создаются для компьютерных платформ, поскольку они обладают высокой вычислительной мощностью и могут использовать сложные алгоритмы обработки видео и движений. Однако такие решения имеют ряд ограничений. Их установка требует соблюдения определенных условий и применения специализированного оборудования (например, камер высокой точности или систем захвата движения), что делает их дорогостоящими и неудобными для использования в повседневной жизни.

Таким образом, несмотря на техническую продвинутость, компьютерные системы полностью не решают проблему доступности перевода жестовой речи. В отличие от них, мобильные приложения способны сопровождать пользователя повсюду, предоставляя возможность для общения в любых условиях — будь то улица, магазин, транспорт или медицинское учреждение. Однако разработка таких программ сталкивается с проблемами, связанными с ограниченными вычислительными ресурсами смартфонов, высокой нагрузкой на систему при работе с видео и необходимостью обучения моделей на больших и разнообразных наборах данных.

Несмотря на сложности, именно мобильное решение способно обеспечить постоянную и доступную коммуникацию для людей с нарушением слуха. Это делает создание такого приложения социально значимым и особенно актуальным. Именно поэтому в рамках выпускной квалификационной работы ставится цель — разработка мобильного приложения для распознавания жестов русского жестового языка (РЖЯ), направленного на улучшение взаимодействия между неслышащими и слышащими людьми.

ГЛАВА 1. РУССКИЙ ЖЕСТОВЫЙ ЯЗЫК: СТРУКТУРА И ТЕХНОЛОГИИ

Для начала необходимо изучить структуру жестового языка, визуальные и пространственные особенности, а также проанализировать существующие программные решения.

1.1. Обзор русского жестового языка

Русский жестовый язык является основным средством общения глухих и слабослышащих в России. Его структура – включая грамматику и построение фраз – существенно отличается от устного русского языка, привычного слышащим. Помимо РЖЯ, в общении применяются вспомогательные средства: тактильная азбука и калькирующая жестовая речь.

РЖЯ, или же жестовая речь, состоит из отдельных жестов, обозначающих слова или целые понятия. Каждое действие формируется из пяти компонентов: четыре из них – мануальные (форма кисти и пальцев, ориентация рук, место исполнения, характер и направление движения), а пятый, немануальный, включает мимику, движения губ, головы и корпуса [9]. Эти составляющие работают как единая система, и изменение любого из них может изменить значение жеста. Например, один и тот же жест, выполненный в разных точках пространства или с иной формой кисти, будет интерпретироваться по-разному.

В рамках текущей разработки основное внимание уделяется мануальным компонентам, так как они являются ключевыми при распознавании отдельных слов. Немануальные компоненты исключены из анализа, поскольку они в большей степени влияют на эмоциональную окраску речи, чем на лексическое значение жеста. В перспективе эти данные можно использовать для реализации функционала перевода целых предложений.

Одной из ключевых особенностей РЖЯ является пространственная грамматика [1]. Пространство перед говорящим используется для установления связи

между объектами, указания на предмет, формирования временных линий и других функций. Это означает, что один и тот же жест, выполненный в разных точках пространства, может иметь различное значение.

Дактильная азбука используется для передачи имен собственных, названий или слов, не имеющих устоявшегося жестового эквивалента. Она представляет собой систему ручных знаков, где каждой букве русского алфавита соответствует определенное положение пальцев одной руки. Всего азбука включает 33 знака.

Калькирующая жестовая речь (КЖЯ) – форма коммуникации, которая использует жесты из РЖЯ, но выстраивает их в соответствии с грамматической структурой звучащего русского языка. КЖЯ часто применяют сурдопереводчики для дословного перевода устных или письменных текстов.

С технической точки зрения, распознавание жестов представляет собой комплексную задачу, включающую:

- обнаружение рук в видеопотоке
- определение ключевых точек кисти
- отслеживание движения точек во времени
- классификацию последовательностей движений как конкретных жестов

Подробный анализ технологий и методов, применяемых для решения этих задач, будет рассмотрен в следующих главах.

1.2. Анализ существующих решений

Прежде чем приступить к разработке собственного приложения, необходимо изучить существующие программные решения. Уже создано много проектов для перевода РЖЯ, но почти все они реализованы в виде настольных или серверных систем, рассчитанных на работу через компьютер. Такие приложения трудно использовать в повседневной жизни из-за отсутствия мобильности.

При изучении магазинов мобильных программ для разных операционных систем было обнаружено, что основную массу инструментов, связанных с жестовыми языками, составляют электронные словари и обучающие программы. Сегмент приложений для перевода ЖЯ в реальном времени представлен крайне ограниченно. В связи с этим в табл.1.1 приведено сравнение разрабатываемого приложения и существующих аналогов по основным критериям.

Таблица 1.1

Сравнение аналогов с разрабатываемым приложением

Критерии	Sign Language AI	Мой РЖЯ	SLAIT	Разрабатываемое приложение
Целевой язык	ASL	РЖЯ	ASL	РЖЯ
Основное назначение	Перевод с языка жестов в реальном времени / Словарь	Перевод с языка жестов в реальном времени	Перевод с языка жестов в реальном времени	Перевод с языка жестов в реальном времени
Тип приложения	Мобильное	Мобильное	Веб-приложение	Мобильное
Технология распознавания	Компьютерное зрение	Сурдопереводчик	Компьютерное зрение	Компьютерное зрение

Среди немногочисленных мобильных приложений выделяется Sign Language AI. Проект позволяет переводить жесты, показанные через камеру смартфона, в реальном времени. Кроме того, приложение содержит собственный словарь с видеозаписями жестов, по которым можно обучиться языку. Это приложение прекрасно подойдет для коммуникации в Америке, так как создано для американского жестового языка (ASL). По этой же причине оно бесполезно для слабослышащих из России.

Что касается русскоязычных мобильных приложений – существует программа «Мой РЖЯ», но его функционал принципиально отличается от концепций разрабатываемого решения и Sign Language AI. Данное приложение представляет собой сервис видеозвонков, соединяющий пользователя с

профессиональными сурдопереводчиками. Хотя такой подход эффективен для обеспечения коммуникации, существует необходимость привлечения профессиональных переводчиков и, как следствие, возможны ограничения по времени доступности сервиса и его стоимость.

Похожим функционалом обладает иностранный веб-сервис SLAIT: он выглядит как приложение для видеозвонков, но направлен на построение коммуникации между организациями и их клиентами, общение которых происходит с помощью ASL. Проект также не поддерживается на мобильных устройствах.

Таким образом, на русском рынке отсутствует прямой аналог разрабатываемому приложению. Особенность проекта заключается в объединении РЖЯ, технологий компьютерного зрения для автоматического перевода и мобильного формата. Следующим логическим шагом является переход от обоснования концепции к выбору инструментария для практической реализации.

ГЛАВА 2. ВЫБОР СРЕДСТВ РЕАЛИЗАЦИИ И ПОСТАНОВКА ЗАДАЧИ

2.1. Анализ существующих решений

Чтобы разработать систему перевода русского жестового языка, сначала нужно понять, какие технологии для этого подходят. Важно решить, с помощью какого языка создавать приложение, как определять положение рук, какую нейросеть использовать для распознавания жестов и на каких данных её обучать.

2.1.1. Языки программирования

Для разработки мобильных приложений популярными языками программирования являются Java, Kotlin и Swift. Каждый из них обладает своими преимуществами и ограничениями.

Java долгое время была основным языком для разработки Android-приложений. Она использует строгую типизацию и объектно-ориентированный подход. У Java много готовых библиотек и инструментов, что делает её удобной для создания приложений разного уровня сложности. Однако у неё есть и недостатки, и главный из них – слишком многословный синтаксис, из-за чего приходится писать больше кода.

Kotlin был создан компанией JetBrains как современная альтернатива Java. С 2019 года он стал официально рекомендуемым языком для Android-разработки. Kotlin проще и короче в написании, при этом полностью совместим с Java и устраняет многие его недостатки, такие как многословность и отсутствие null-безопасности. Благодаря этому язык программирования Kotlin быстро стал основным выбором разработчиков.

Swift был разработан компанией Apple для создания приложений для iOS, macOS и других продуктов. Он отличается простым и понятным синтаксисом, строгой типизацией и автоматическим управлением памятью. Данный язык

программирования считается современным и удобным инструментом для мобильной разработки.

Согласно статистике за декабрь 2024 года, в России большинство пользователей используют смартфоны на базе Android – их доля составляет около 58–60%, тогда как владельцев iPhone – лишь 40–42% [6]. Поскольку приложение ориентировано прежде всего на жителей Российской Федерации, логичным выбором становится платформа Android, как более массовая. Соответственно, для разработки следует использовать язык, предназначенный именно для этой операционной системы.

Таким образом, выбор делается между Java и Kotlin. В рамках проекта принято решение использовать язык Kotlin, так как он проще в использовании и позволяет писать более лаконичный код. Для создания пользовательского интерфейса будет использоваться Jetpack Compose [16] – набор инструментов от компании Google. Он значительно упрощает разработку UI и позволяет создавать интерфейсы с меньшими затратами кода по сравнению с традиционным подходом с использованием XML.

2.1.2. Инструменты для обработки изображений и детекции рук

После выбора языка нужно определиться со способом распознавания ладоней. Для этого можно использовать библиотеки компьютерного зрения.

OpenCV [19] является одной из наиболее доступных библиотек с открытым исходным кодом. Она не предоставляет специализированных решений для детекции рук, но для этого можно интегрировать предварительно обученные модели или классические методы компьютерного зрения (контурный анализ, каскады Хаара).

TensorFlow Lite представляет собой оптимизированную версию TensorFlow [12] для развертывания моделей машинного обучения на мобильных устройствах. Платформа предоставляет готовые решения для распознавания жестов и

поддерживает создание кастомных моделей с минимальным объемом кода [20]. В целом, TensorFlow Lite направлен на развертывание уже обученных моделей.

OpenPose [13] является первой системой реального времени для многопользовательской детекции тела, рук, лица и ног (всего 135 ключевых точек). Для рук обеспечивается детекция 21 ключевой точки и поддерживается одновременная обработка нескольких человек в кадре. Библиотека предоставляет готовые предварительно обученные модели для Python, C++ и MATLAB. Основными ограничениями OpenPose являются высокие вычислительные требования, делающие его менее подходящим для мобильных устройств.

MediaPipe Hand Landmarker [15], разработанный Google, представляет собой специализированное решение для детекции и отслеживания рук в режиме реального времени. Система использует двухэтапный подход: сначала определяет область руки на изображении, затем распознает 21 трехмерную координату ключевых точек руки. Библиотека предоставляет простые API для Android, iOS, Python и JavaScript и обеспечивает точность 95.7% для детекции ладони [18].

Чтобы определиться с выбором инструментария, необходимо сравнить описанные выше системы. Сравнение отображено в табл. 2.1.

Таблица 2.1

Сравнение инструментов

Критерий	OpenCV	TensorFlow Lite	OnePose	MediaPipe Hand Landmarker
Специализация на руках	Нет, универсальная библиотека	Нет, универсальная платформа	Частично	Да
Поддержка мобильных устройств	Да	Да, оптимизирован	Ограничена	Да, оптимизирован
Готовые API	Базовые функции CV	Да	Python, C++, MATLAB	Android, iOS, Python, JavaScript
Вычислительные требования	Средние	Низкие	Высокие	Низкие

На основе проведенного анализа выбран MediaPipe Hand Landmarker как наиболее подходящее решение, обеспечивающее специализированную работу с жестами рук, высокую точность распознавания и оптимизацию для мобильных платформ. Другие решения требуют дополнительные настройки, в которых выбранный инструмент не нуждается.

2.1.3. Нейронные сети для обучения модели

В основе системы распознавания жестов лежит анализ последовательности движений рук во времени. Каждый жест представляет собой динамический процесс, для изучения которого требуются специализированные архитектуры нейронных сетей, способные учитывать как пространственные характеристики позы рук в каждый момент времени, так и временные зависимости между последовательными кадрами.

LSTM (Long Short-Term Memory) – это разновидности рекуррентных нейронных сетей, спроектированные для работы с данными, где важны длительные зависимости [21]. Благодаря внутренней "памяти" и механизмам управления информацией (входные, выходные и забывающие шлюзы), нейронная сеть способна "запоминать" важные данные на протяжении всей последовательности. Основное достоинство LSTM – высокая способность к обучению на сложных временных паттернах, однако их вычислительная нагрузка может быть значительной, особенно на мобильных устройствах.

GRU (Gated Recurrent Unit)– это упрощённая версия LSTM с меньшим количеством параметров. GRU объединяет входной и "забывающий" шлюзы в один "шлюз обновления", что делает модель более компактной и потенциально ускоряет обучение и применение [14]. Она эффективно справляется с задачами умеренной сложности, хотя в задачах с длинными последовательностями точность может снижаться.

3D-CNN (3-Dimensional Convolutional Neural Networks) [11] добавляют временное измерение к традиционным двумерным свёрточным нейронным сетям.

Это позволяет им одновременно извлекать как пространственные, так и временные признаки из последовательности кадров. Такой подход особенно хорошо работает при распознавании жестов с четко выраженными локальными движениями. Преимущество 3D-CNN – в способности обрабатывать пространственно-временные паттерны как единое целое. Однако эти модели требуют большого объёма данных для обучения и значительных вычислительных мощностей, что может ограничивать их применение на менее производительных устройствах.

В рамках исследования планируется реализация всех трёх вышеупомянутых нейронных сетей. Каждая из рассмотренных моделей обладает уникальными характеристиками и преимуществами для конкретных аспектов распознавания жестов. В главе, посвященной тестированию, будет проведён экспериментальный анализ каждой архитектуры на наборе данных, включающий оценку точности распознавания и времени обучения.

2.1.4. Набор данных для обучения нейросетевой модели

Чтобы нейронные сети справились с обучением и показали хороший результат, нужны качественные данные для обучения. Так как целью является распознавание динамичных жестов, нужно подобрать набор данных, состоящий из видеозаписей.

Наилучшим выбором является SLOVO Dataset [17] – большой датасет русского жестового языка от компании Сбер. Он включает в себя 20 тысяч высококачественных видеозаписей, разделенных на тысячу классов отдельных жестов РЖЯ. В создании датасета приняли участие 194 носителя жестового языка, благодаря чему нейронная сеть может обучиться распознаванию движений с различными стилями исполнения. Кроме того, SLOVO содержит предварительно обработанные координаты ключевых точек рук, полученных с помощью Mediapipe. Это является решающим критерием при выборе датасета, так как для определения рук пользователя также будет использоваться данный фреймворк.

Для начала решено реализовать ограниченную версию системы, способную распознавать 30 случайных жестов из датасета SLOVO. В дальнейшем планируется постепенное расширение словаря до полного объема датасета SLOVO.

2.2. Постановка задачи и требований

Определение четких и структурированных требований является важным этапом разработки любого мобильного приложения. Требования необходимы для проектирования архитектуры системы, выбора технологических решений, планирования разработки и последующего тестирования функциональности.

2.2.1. Функциональные требования

Аутентификация и управление пользователями

- 1) Приложение должно предоставлять возможность регистрации новых пользователей с указанием имени, электронной почты и пароля.
- 2) Приложение должно поддерживать автоматический вход в систему для авторизованных пользователей при повторном запуске.

Распознавание жестов

- 1) Приложение должно осуществлять захват видеопотока с камеры устройства в режиме реального времени для анализа жестов рук.
- 2) Система должна автоматически определять ключевые точки рук (21 точка на каждую руку) с использованием технологии MediaPipe.
- 3) Приложение должно поддерживать распознавание жестов как с фронтальной, так и с задней камеры устройства.

Обработка и анализ жестов

- 1) Система должна фильтровать некачественные данные и пустые кадры для повышения точности распознавания.
- 2) Приложение должно определять фазы выполнения жеста (инициализация, движение, завершение) для оптимального момента распознавания.

Результаты распознавания

- 1) Система должна отображать распознанный жест в виде текста на экране с указанием уровня уверенности модели.
- 2) Приложение должно поддерживать распознавание 30 жестов РЖЯ, указанные на рис. 2.1.

Абстрактные понятия	Местоимения	Буквы	Глаголы
способ	нас	ё	ждать
сторона	я	с	переводить
шум	тот		

Наречия	Характеристики	Предметы/материалы	Числа
довольно	безопасный	бумага	восемьдесят
особенно	нежный	вода	одиннадцать
только	хороший	воздух	шестнадцать
несколько	следующий	газета	тысяча
	последний		

Рис.2.1. Слова для распознавания

2.2.2. Нефункциональные требования

Производительность

- 1) Время отклика системы на выполнение жеста не должно превышать 3 секунд от момента завершения жеста до отображения результата.
- 2) Приложение должно обрабатывать видеопоток с частотой не менее 15–20 кадров в секунду для плавного отображения жестов.

Надежность

- 1) Система должна корректно обрабатывать временные потери соединения с сервером и автоматически восстанавливать подключение.

- 2) Приложение должно устойчиво работать при переключении между камерами без потери данных или сбоев.
- 3) Система должна корректно обрабатывать ситуации отсутствия рук в кадре без возникновения ошибок.

Удобство использования

- 1) Интерфейс приложения должен быть интуитивно понятным пользователям различного уровня технической подготовки.
- 2) Система должна предоставлять визуальную обратную связь о качестве захвата жестов через отображение ключевых точек рук.
- 3) Приложение должно включать контекстную справку и инструкции по использованию для новых пользователей.

Совместимость

- 1) Приложение должно работать на устройствах, использующих операционную систему Android 8.0 и выше
- 2) Приложение не должно конфликтовать с другими программами
- 3) Интерфейс системы должен адаптироваться под различные размеры экрана

ГЛАВА 3. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ СИСТЕМЫ

3.1. Выбор архитектуры системы

Для разработки системы перевода жестов для мобильного приложения необходимо выбрать такую архитектуру, которая сможет обеспечить как эффективное распознавание, так и удобное взаимодействие пользователя с приложением. Для этого отлично подходит клиент-серверная архитектура, которая предполагает разделение функциональности между клиентской (мобильным приложением) и серверной (системой обработки и распознавания жестов) частями.

Основной альтернативой клиент-серверной архитектуры является выполнение всех вычислений непосредственно на мобильном устройстве. Однако у такого подхода есть существенные недостатки. Алгоритмы распознавания жестов, основанные на глубоких нейронных сетях, требуют немалых вычислительных ресурсов. Запуск ресурсоемких задач на мобильных устройствах, даже самых мощных, приводит к замедлению работы приложения, сильному нагреванию устройства и быстрой разрядке батареи. Кроме того, обновления модели машинного обучения, необходимые для добавления новых жестов, потребовало бы полного обновления всего приложения пользователем, что затрудняет оперативное внедрение улучшений. Размер самих моделей также может стать проблемой для хранения на устройстве.

Все описанные выше ограничения делают клиент-серверную архитектуру предпочтительной. Перенос ресурсоемких вычислений по распознаванию на выделенный сервер позволяет снять нагрузку с мобильного устройства. Обновление моделей машинного обучения происходит централизованно на сервере, что не заметно для обычного пользователя и позволяет разработчикам легко развертывать улучшения.

3.2. Проектирование клиента

3.2.1. Проектирование пользовательского интерфейса

От качества и интуитивности интерфейса приложения напрямую зависит опыт взаимодействия пользователя с системой. При разработке интерфейса следует учитывать как функциональные требования, так и современные принципы UX/UI дизайна для мобильных приложений.

В основе дизайна приложения лежит концепция Material Design от Google, которая обеспечивает гармоничное сочетание элементов управления и содержательной части. Приложение поддерживает как светлую, так и темную тему оформления, синхронизируясь с темой устройства.

На рис. 3.1 показан спроектированный интерфейс в Figma, состоящий из 4 экранов: начального, регистрации, авторизации и с функционалом распознавания жестов.

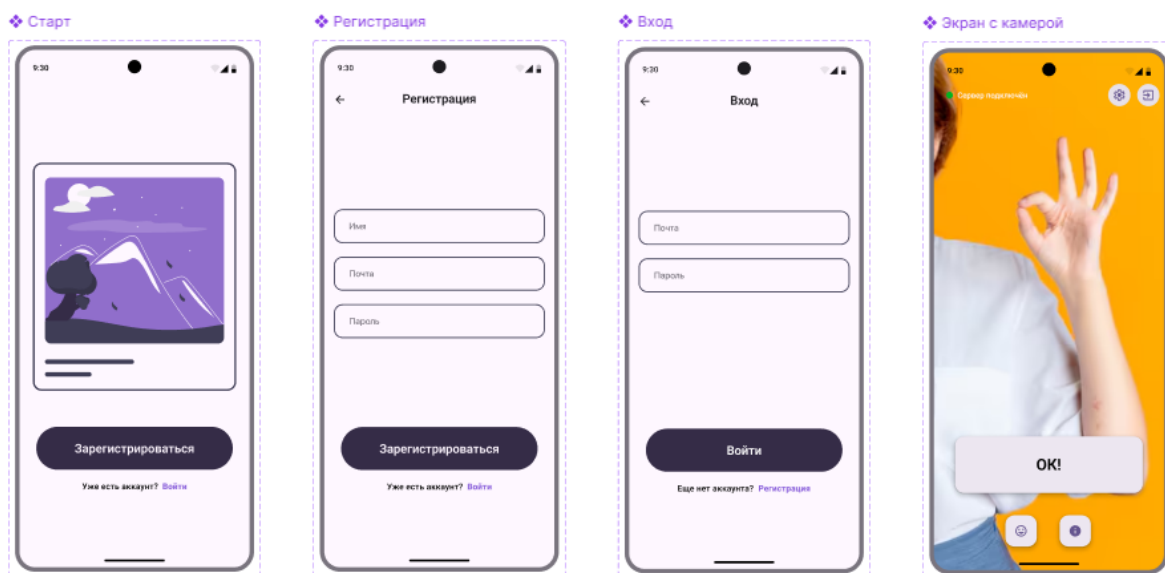


Рис.3.1. Пользовательский интерфейс

Начальный экран содержит в себе кнопки авторизации и регистрации. Экран регистрации имеет поля для ввода имени, электронной почты и пароля, а также соответствующую кнопку для подтверждения регистрации. Если

пользователь ошибся экраном, он может перейти сразу ко входу по соответствующей кнопке.

Экран авторизации похож на предыдущий экран, но для входа в систему достаточно просто указать почту и пароль. Нажав на кнопку «Войти», пользователь перейдет на главный экран, если он уже был зарегистрирован в системе.

На основном экране после разрешения на использование камеры сразу же начинается захват видеопотока. Сверху присутствует индикатор о состоянии сервера. Если сервер неактивен – результат распознавания жеста не отобразится на экране. Также присутствуют кнопки выхода из системы и настройки для разработчика, которые в дальнейшем будут скрыты для обычного пользователя. Они содержат в себе настройку IP-адреса и порта для связи с сервером. Снизу присутствуют кнопки вызова справки и смены вида камеры. Кнопки справки, настроек и выхода вызывают диалоговые окна.

Если в кадр попали кисти рук и сервер запущен, то программа начнет распознавание, результат будет показан в блоке над нижними кнопками, как показано на последнем экране (см. рис. 3.1).

3.2.2. Проектирование архитектуры клиента

Клиентская часть приложения разрабатывается для ОС Android с использованием современных инструментов. Для обеспечения гибкости выбран архитектурный паттерн MVVM (Model-View-ViewModel). Данный паттерн разграничивает зоны ответственности компонентов:

- View (представление) отвечает исключительно за отображение пользовательского интерфейса (UI) и передачу действий пользователя в ViewModel. Не содержит логики обработки данных или управления состоянием
- ViewModel (модель представления) выступает посредником между View и Model. Хранит и управляет состоянием UI, обрабатывает команды от View и запрашивает/отправляет данные через Model. Не имеет прямой ссылки на View, что обеспечивает независимость компонентов

- Model (модель) отвечает за бизнес-логику приложения и доступ к данным (из сети, базы данных, локальных файлов)

Архитектура должна быть организована в виде логически связанных между собой слоев, каждый из которых отвечает за определенные функции.

Слой View охватывает все компоненты, с которыми будет взаимодействовать пользователь. В него входят стартовый экран, экраны аутентификации, а также основной интерфейс работы с камерой. Дополнительно будут реализованы диалоговые окна для настройки подключения к серверу, отображения справки и подтверждения действий. Важной частью этого слоя является компонент визуализации, который накладывает ключевые точки распознанных рук поверх видеопотока в реальном времени.

Слой ViewModel берет на себя управление состоянием пользовательского интерфейса и бизнес-логикой. Один из его ключевых компонентов отвечает за аутентификацию: обработку регистрации, входа, управления сессией и взаимодействие с серверным API. Второй компонент координирует процесс обработки видеопотока, обеспечивая связь между действиями пользователя в интерфейсе и утилитарными сервисами распознавания.

Навигационный слой ответственен за логику переходов между экранами. В его основе лежит система маршрутизации, определяющая доступные пути навигации. Механизм управления переходами реализован таким образом, чтобы учитывать текущий статус аутентификации пользователя, не допуская неавторизованный доступ к защищенным экранам.

Слой Model организован в виде трех взаимосвязанных подсистем для работы с данными. Модели данных представляют структуры пользователя, ответов от сервера и запросов. API сервисы абстрагируют всю работу с сетью, инкапсулируя логику запросов к API аутентификации и распознавания. Конфигурационная подсистема позволяет гибко управлять настройками подключения, включая быстрое переключение между серверами для разработки. В дальнейшем

настройки можно ограничить для определенного круга пользователей, либо убрать совсем.

Локальное хранилище обеспечивает безопасное сохранение токенов аутентификации и данных сессии. Для защиты конфиденциальной информации предполагается использовать зашифрованные механизмы хранения.

Блок с утилитами содержит низкоуровневые компоненты для работы с аппаратным обеспечением и обработки данных. Менеджер камеры абстрагирует работу с системным API, управляет жизненным циклом камеры и оптимизирует захват кадров. Обработчик изображений интегрирован с библиотеками машинного обучения для нахождения ключевых точек рук в каждом кадре. Ключевым элементом является анализатор жестов. Его задача — реализовать алгоритм временного анализа последовательностей кадров. Он определяет начало и конец жеста, отбраковывает лишние кадры и формирует оптимальные наборы данных для отправки на сервер.

Взаимодействие организовано через четко определенные потоки. Пользователь инициирует действия в слое View, который передает их в ViewModel. Далее ViewModel координирует операции, обращаясь к Model для работы с данными (например, для входа в систему) или к блоку с утилитами для запуска процесса распознавания.

Цепочка обработки жестов выглядит так: Менеджер камеры предоставляет кадры, Обработчик изображений извлекает из них ключевые точки, Анализатор жестов формирует из последовательности точек готовый жест и передает его API сервису в слое Model. API сервис отправляет данные на сервер, получает результат и через ViewModel возвращает его в View для отображения пользователю.

Описанная архитектура клиента изображена на рис. 3.2.

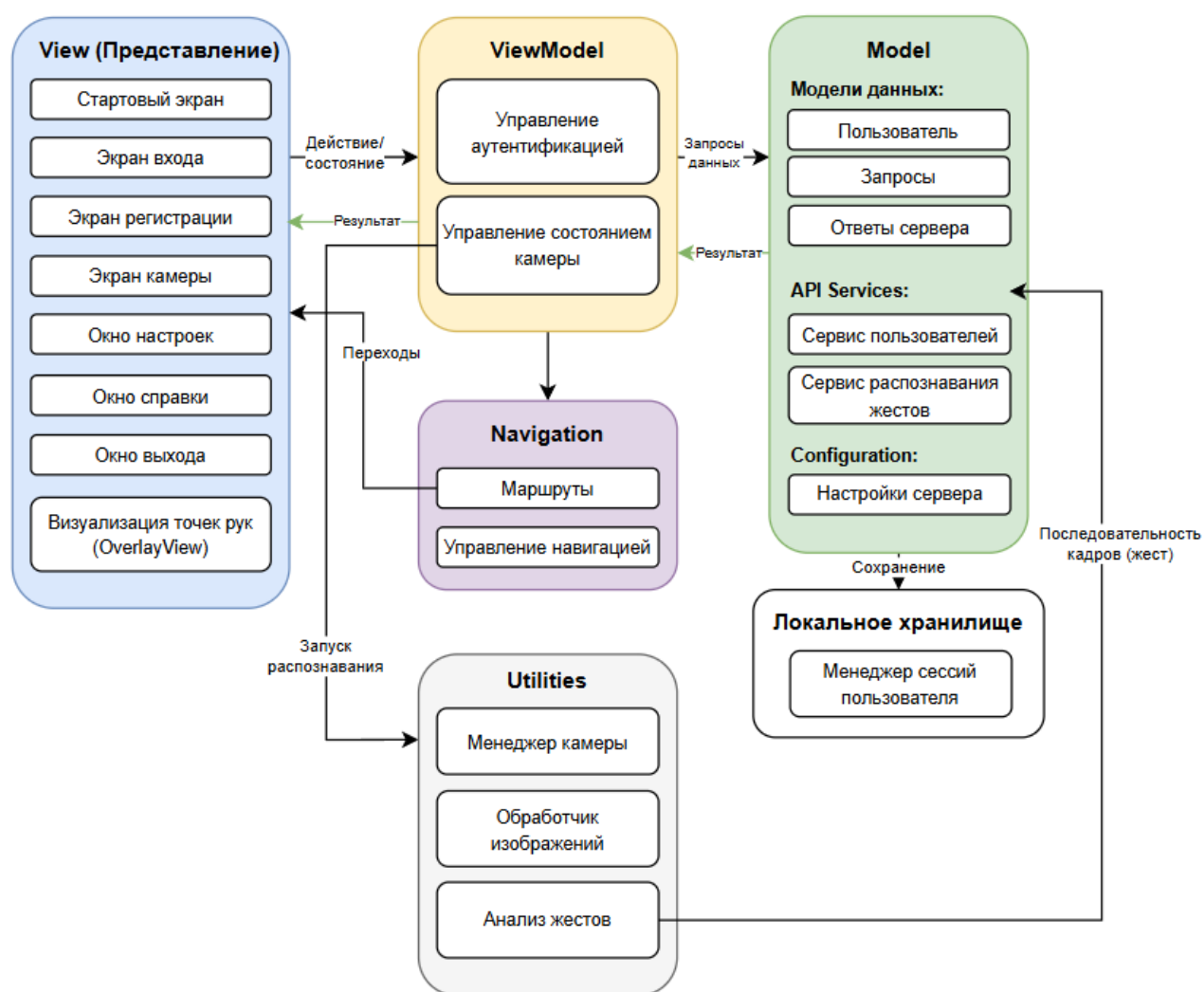


Рис. 3.2. Архитектура клиента

3.3. Проектирование сервера

Серверная часть системы распознавания языка жестов реализована в рамках модульной монолитной архитектуры. Такой подход выбран, чтобы снизить задержки при обработке жестов в реальном времени и упростить запуск и поддержку системы. Вся логика определения и анализа жестов вынесена на клиентскую сторону, что позволяет серверу сосредоточиться исключительно на распознавании готовых последовательностей.

Взаимодействие с клиентскими приложениями осуществляется через REST API. Он предоставляет эндпоинты для аутентификации пользователей и

синхронной обработки готовых последовательностей жестов с немедленным возвратом результатов.

Сервис аутентификации реализован как внутренний модуль, отвечающий за полный цикл управления пользователями. Для этого созданы эндпоинты: `/api/register`, `/api/login`, `/api/logout` и `/api/user`. В задачи модуля входит валидация входных данных, хеширование паролей, генерация и проверка токенов, а также взаимодействие с базой данных пользователей.

Сервис распознавания жестов является ядром приложения. Его функциональность доступна через единственный эндпоинт `/features`, который принимает готовые последовательности жестов и немедленно возвращает результаты распознавания. Модуль выполняет синхронную обработку:

- Процессор жестов выполняет валидацию входных данных и немедленное распознавание полученных последовательностей
- Управление моделями предоставляет доступ к предварительно загруженным нейронным сетям

В отличие от традиционной архитектуры с накоплением кадров и асинхронной обработкой, данная система исключает фоновые потоки, очереди и промежуточные буферы. Вся обработка происходит синхронно в рамках одного HTTP-запроса, обеспечивая прямую передачу от API к процессору жестов и немедленный возврат результата. Архитектура описанных процессов сервера изображена на рис.3.3.

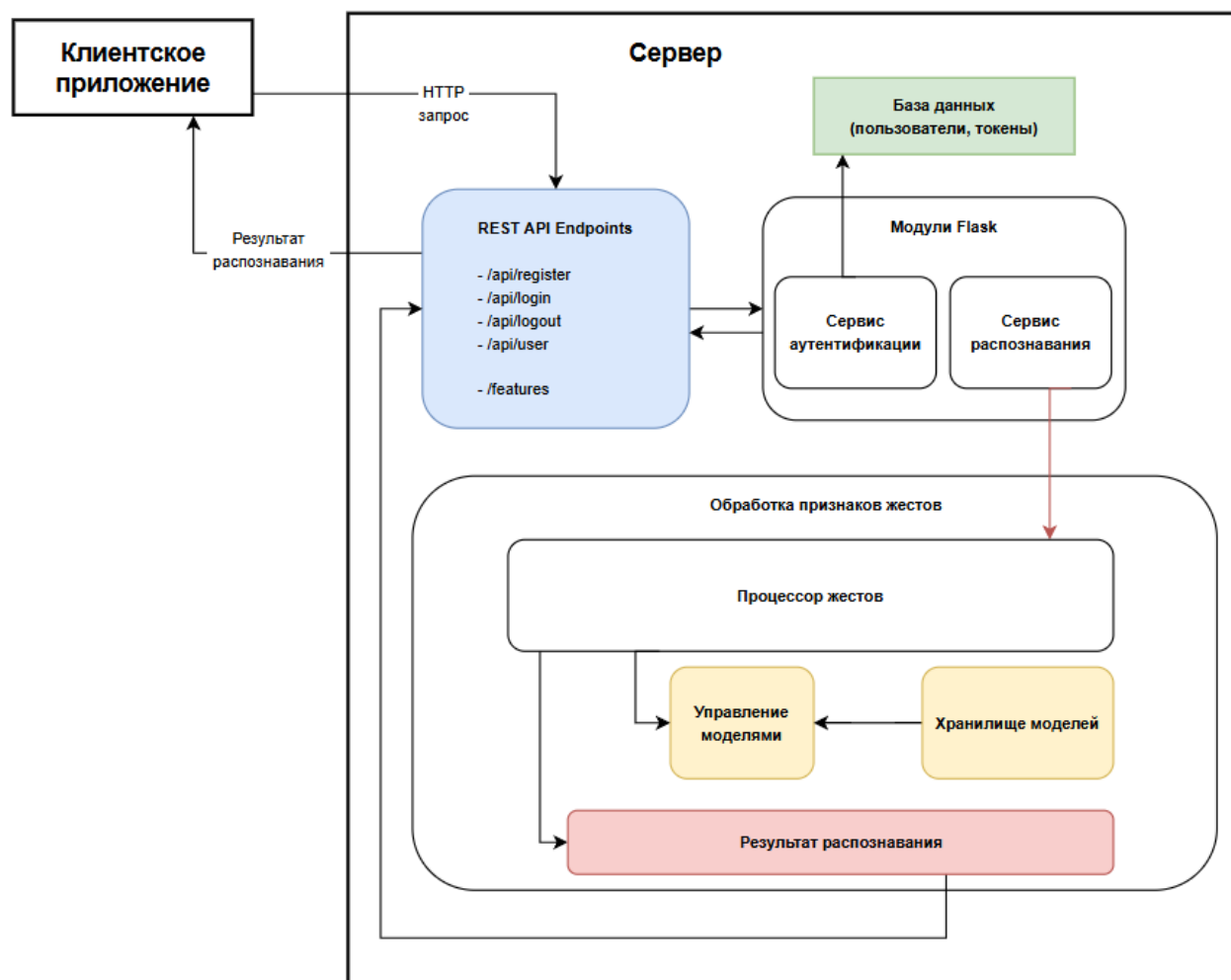


Рис.3.3. Архитектура сервера

Особое внимание следует уделить процессу обработки запросов на распознавание жестов. Он представляет собой упрощенный синхронный конвейер, где вся сложная логика анализа жестов выполняется на клиентской стороне.

Процесс инициируется на клиентском приложении: анализатор жестов определяет завершение жеста, формирует оптимальную последовательность из 20 кадров (2520 признаков) и отправляет её HTTP POST-запросом на эндпоинт `/features`. На серверной стороне запрос принимается и обрабатывается синхронно:

- 1) Полученная последовательность признаков проверяется на корректность (2520 признаков) и качество данных

- 2) Данные преобразуются в формат тензора, подходящий для модели машинного обучения.
- 3) Управление моделями предоставляет доступ к предварительно загруженной нейронной сети. Модель загружается один раз при запуске сервера и остается в памяти для быстрого доступа.
- 4) Процессор жестов передает подготовленные данные в модель машинного обучения. Модель анализирует последовательность признаков и возвращает вектор вероятностей для каждого возможного класса жестов.
- 5) Процессор жестов анализирует вектор вероятностей, находит жест с максимальной вероятностью, проверяет превышение порога уверенности и сопоставляет результат с текстовой меткой жеста.
- 6) Финальный результат, содержащий название жеста, уровень уверенности и дополнительные метаданные, немедленно возвращается клиенту в JSON-формате в рамках того же HTTP-ответа.

Такая синхронная модель взаимодействия обеспечивает минимальную задержку (50-150мс) и упрощает логику как серверной, так и клиентской части. Клиентское приложение получает результат сразу после отправки данных и может немедленно обновить пользовательский интерфейс.

3.4. Проектирование базы данных

База данных системы должна хранить информацию о пользователях и их сессиях. Для регистрации пользователь должен ввести email, имя и пароль. Соответствующие данные должны сохраниться в базе данных, чтобы в дальнейшем по ним можно было снова войти в систему. Концептуальная модель включает сущности, указанные на рис.3.4.

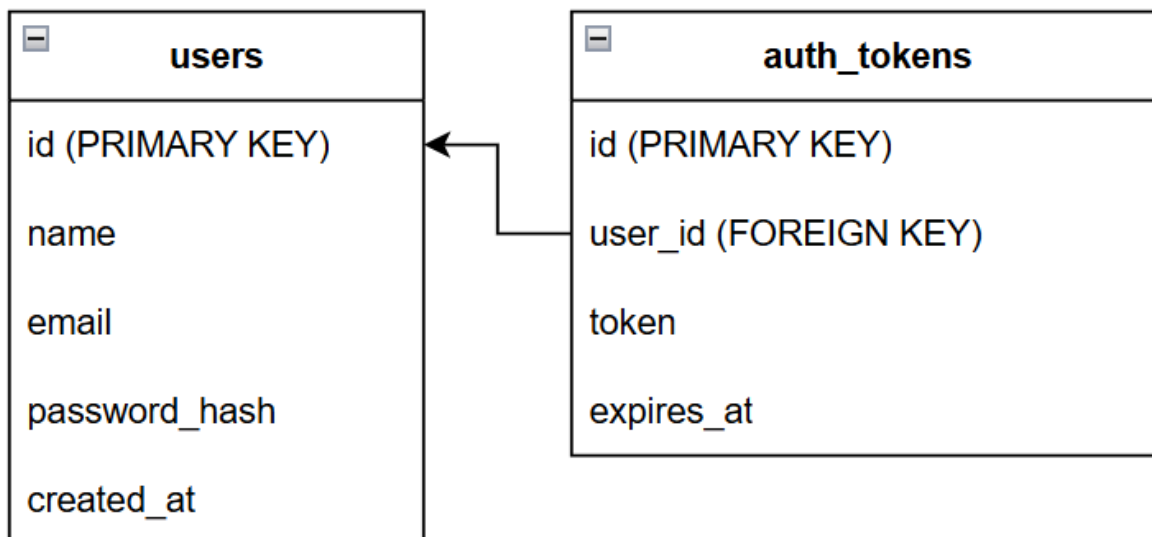


Рис.3.4. ER-диаграмма базы данных

Основой системы аутентификации является таблица `users`. Она включает `id` (первичный ключ), `name` (отображаемое имя), `email` (уникальный логин) и `created_at` (время создания). Важнейшее поле `password_hash` содержит хеш пароля, что обеспечивает защиту данных пользователя даже при утечке информации из базы.

Таблица `auth_tokens` реализует механизм сессий. Она связана с пользователем через `user_id` (внешний ключ). Поле `token` хранит уникальную строку, которая временно идентифицирует пользователя, избавляя от необходимости постоянно вводить пароль. Для безопасности у каждого токена есть `expires_at` – время истечения, по которому он становится недействительным. Это снижает риски, связанные с компрометацией токенов.

Между таблицами установлена связь «один-ко-многим». Это значит, что один пользователь может иметь несколько активных сессий одновременно (например, на телефоне и в браузере), но каждый токен принадлежит строго одному пользователю. Такая структура обеспечивает надёжную и безопасную аутентификацию, а её простота облегчает поддержку и развёртывание системы.

Теперь, когда вся система спроектирована, можно приступить к программной реализации.

ГЛАВА 4. РЕАЛИЗАЦИЯ СИСТЕМЫ РАСПОЗНАВАНИЯ ЖЕСТОВ

На основе спроектированной в предыдущей главе архитектуры выполнена практическая реализация системы распознавания жестов РЖЯ.

4.1. Разработка клиентской части

Клиентская часть приложения реализована для платформы Android с использованием Kotlin как основного языка программирования и Jetpack Compose для построения пользовательского интерфейса.

4.1.1. Структура проекта

Проект клиентской части организован согласно паттерну MVVM (Model-View-ViewModel) и состоит из следующих ключевых модулей:

Слой представления (View)

- screen/ – экраны приложения с использованием Jetpack Compose
- dialog/ – диалоговые окна для настроек и справки
- OverlayView.kt – компонент для визуализации ключевых точек рук

Слой бизнес-логики (ViewModel)

- viewmodel/AuthViewModel.kt – управление аутентификацией
- viewmodel/CameraViewModel.kt – координация работы с камерой

Слой данных (Model)

- api/ – сервисы для взаимодействия с сервером
- data/UserSessionManager – модели данных и менеджер сессий
- hands/HandLandmarkHelper – компоненты для работы с MediaPipe

Утилиты и вспомогательные компоненты

- camera/CameraHelper.kt – управление камерой устройства

- navigation/ – компоненты навигации между экранами
- GesturePhaseDetector.kt – анализатор фаз жестов

4.1.2. Захват и обработка видеопотока с камеры

Основой работы системы является непрерывный захват видеопотока с камеры устройства. За это отвечает класс `CameraHelper`, который инкапсулирует всю логику работы с камерой Android и предоставляет упрощенный интерфейс для остальных компонентов системы. Полный код класса отображен в приложении 1 в листинге 1.

При инициализации `CameraHelper` автоматически определяет доступные камеры устройства и выбирает оптимальную конфигурацию. Система поддерживает работу как с фронтальной, так и с задней камерой. Переключение между камерами реализовано через метод `toggleCamera`. Он выполняет корректное завершение работы текущей камеры и инициализацию новой с сохранением всех настроек обработки.

Для обеспечения стабильной работы `CameraHelper` использует собственный исполнительный поток (`cameraExecutor`). Он предотвращает блокировку основного UI-потока при выполнении операций с камерой. Класс также управляет жизненным циклом камеры, автоматически освобождая ресурсы при завершении работы приложения или переходе в фоновый режим.

Особенностью реализации является настройка анализатора изображений через класс `ImageAnalyzer`. Этот компонент получает каждый кадр из видеопотока и передает его для дальнейшей обработки. Используется также стратегия, которая обеспечивает обработку только самых свежих кадров, предотвращая накопление задержек при высокой частоте кадров.

4.1.3. Детекция ключевых точек рук с `MediaPipe`

Распознавание положения рук в каждом кадре выполняется с помощью класса `HandLandmarkHelper`, который является оберткой над библиотекой

MediaPipe Hand Landmarker. Этот компонент настраивается для работы в режиме реального времени и может одновременно отслеживать до двух рук в кадре. Код класса представлен в приложении 2 в листинге 2.

При получении нового кадра из CameraHelper происходит сложная цепочка обработки. Сначала изображение преобразуется из внутреннего формата Android (YUV) в формат, понятный MediaPipe. Затем применяются необходимые трансформации: поворот изображения в соответствии с ориентацией устройства и зеркальное отражение для фронтальной камеры, чтобы обеспечить естественное восприятие пользователем.

HandLandmarkHelper возвращает результаты через callback-интерфейс LandmarkListener. Для каждой обнаруженной руки система получает массив из 21 ключевой точки, где каждая точка содержит трехмерные координаты (x, y, z). Эти координаты нормализованы относительно размеров изображения, что обеспечивает независимость от разрешения камеры. Если в кадре руки не обнаружены или качество изображения недостаточно для надежного распознавания, система корректно обрабатывает такие ситуации, не прерывая общий процесс работы.

4.1.4. Подготовка данных для передачи на сервер

Координаты ключевых точек, полученные от MediaPipe, требуют дополнительной обработки перед отправкой на сервер. За это отвечает функция extractFeaturesFromLandmarks, которая выполняет несколько важных преобразований. Код представлен в приложении 3 в листинге 3.

Во-первых, система нормализует координаты для фронтальной камеры, применяя зеркальное отражение по оси X ($1.0 - x$), чтобы обеспечить единообразную обработку данных независимо от используемой камеры. Это важно для корректной работы модели машинного обучения. Во-вторых, для повышения вариативности данных и предотвращения проблем с абсолютно идентичными кадрами применяется добавление минимального случайного шума к координатам.

Шум настолько мал, что не влияет на точность распознавания, но помогает избежать артефактов при обработке.

Система формирует единый вектор признаков размерностью 126 элементов: по 63 элемента для каждой руки ($21 \text{ точка} \times 3 \text{ координаты}$). Если одна из рук не видна в кадре, соответствующие позиции заполняются нулевыми значениями с добавлением минимального шума.

Ключевым компонентом обработки является `GesturePhaseDetector` - интеллектуальный анализатор жестов, который определяет фазы выполнения жеста. Он отслеживает инициализацию жеста, активную фазу движения, стабилизацию и завершение. Благодаря этому система может точно определить момент, когда жест полностью выполнен и готов для распознавания. Код класса отображен в приложении 4 в листинге 4.

`GesturePhaseDetector` накапливает последовательность из 20 кадров, анализируя движение между соседними кадрами. При завершении жеста он выбирает наиболее информативные кадры из собранной последовательности и формирует оптимальный набор данных для отправки на сервер.

4.1.5. Визуализация ключевых точек рук

Для обеспечения визуальной обратной связи с пользователем реализован компонент `OverlayView`, который отображает распознанные ключевые точки рук поверх видеопотока камеры. Этот класс наследуется от стандартного `View` `Android` и переопределяет метод `draw` для кастомной отрисовки. Пример визуализации точек изображен на рис.4.1.

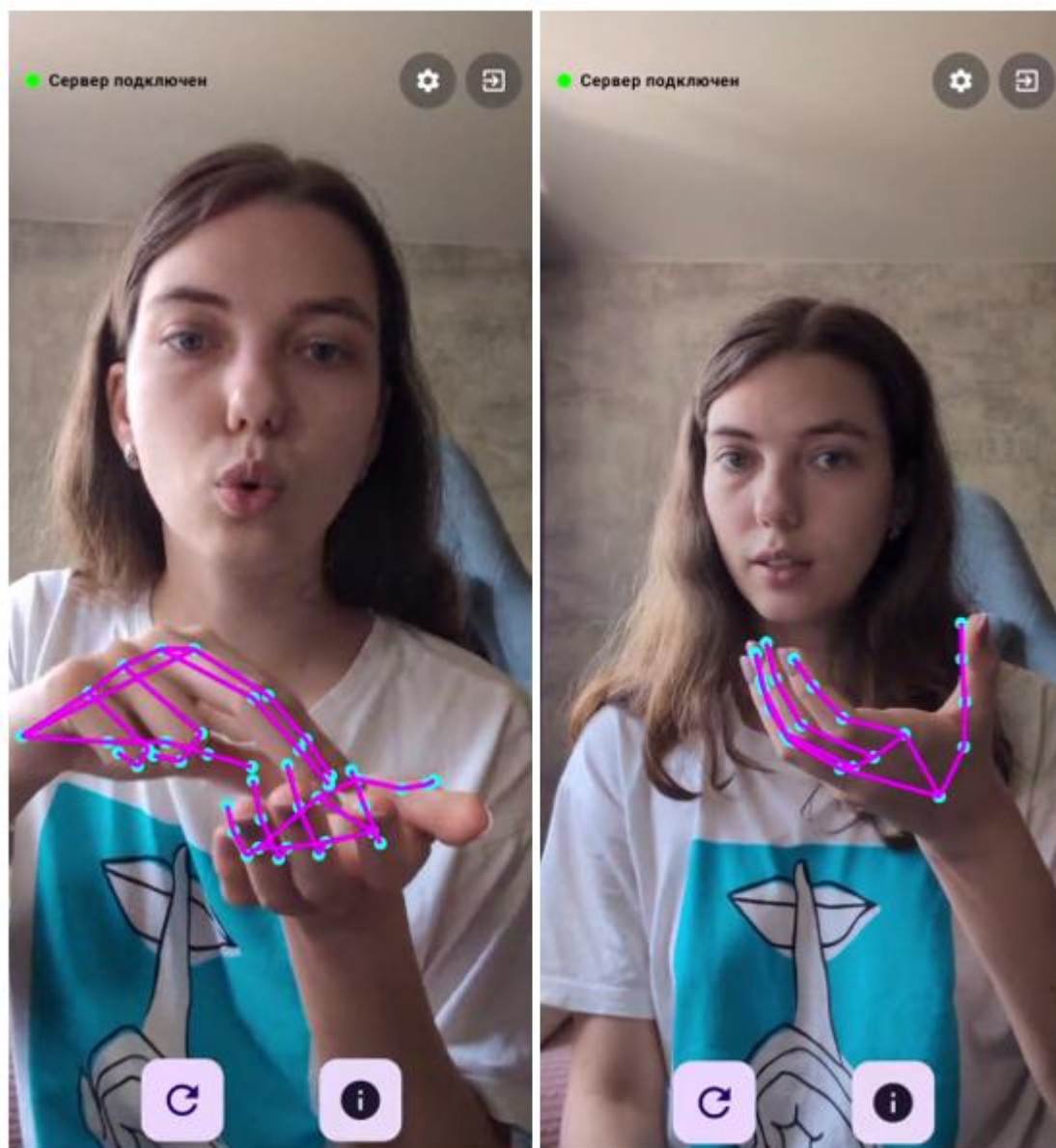


Рис.4.1. Визуализация ключевых точек рук

Основная сложность реализации OverlayView заключается в корректном масштабировании и позиционировании точек. Координаты, полученные от MediaPipe, нормализованы относительно размеров изображения камеры, но изображение на экране может иметь другие пропорции. Метод `calculateScaleFactor` вычисляет необходимые коэффициенты масштабирования и смещения для точного наложения визуализации

Для каждой ключевой точки руки OverlayView рисует цветную окружность с помощью объекта `Paint`. Дополнительно отображаются соединения между точками, формирующие "скелет" руки. Используется предопределенный набор

соединений `HandLandmarker.HAND_CONNECTIONS`, который определяет, какие точки должны быть соединены линиями.

Визуализация обновляется синхронно с обработкой кадров. Метод `setResults` получает новые результаты распознавания и вызывает `invalidate()`, что запускает перерисовку `overlay`. Это обеспечивает плавное отображение движения рук в реальном времени. Полный код класса представлен в приложении 5 на листинге 5.

4.1.6. Отображение результатов распознавания

Система обмена данными с сервером построена на принципе синхронного взаимодействия - клиент отправляет данные жеста и немедленно получает результат распознавания в рамках одного HTTP-запроса. За это отвечает класс `SignLanguageApiService`, который инкапсулирует всю логику сетевого взаимодействия. Полный код представлен в приложении 6 в листинге 6.

Когда `GesturePhaseDetector` определяет завершение жеста, последовательность кадров отправляется на сервер через метод `recognizeGesture`. Этот метод формирует HTTP POST-запрос с JSON-телом, содержащим массив признаков, и ожидает результат распознавания в том же HTTP-ответе.

Полученные от сервера результаты содержат название распознанного жеста, уровень уверенности модели и идентификатор класса. Эта информация передается в пользовательский интерфейс через `reactive state management` с использованием `mutableStateOf`.

Для отображения результата используется компонент `DetectedGestureCard` - карточка `Material Design`, которая появляется в нижней части экрана при успешном распознавании жеста. Карточка показывает название жеста и процент уверенности. Через несколько секунд карточка автоматически исчезает, готовая к

отображению следующего результата. Пример отображения ключевых точек показан на рис. 4.2.

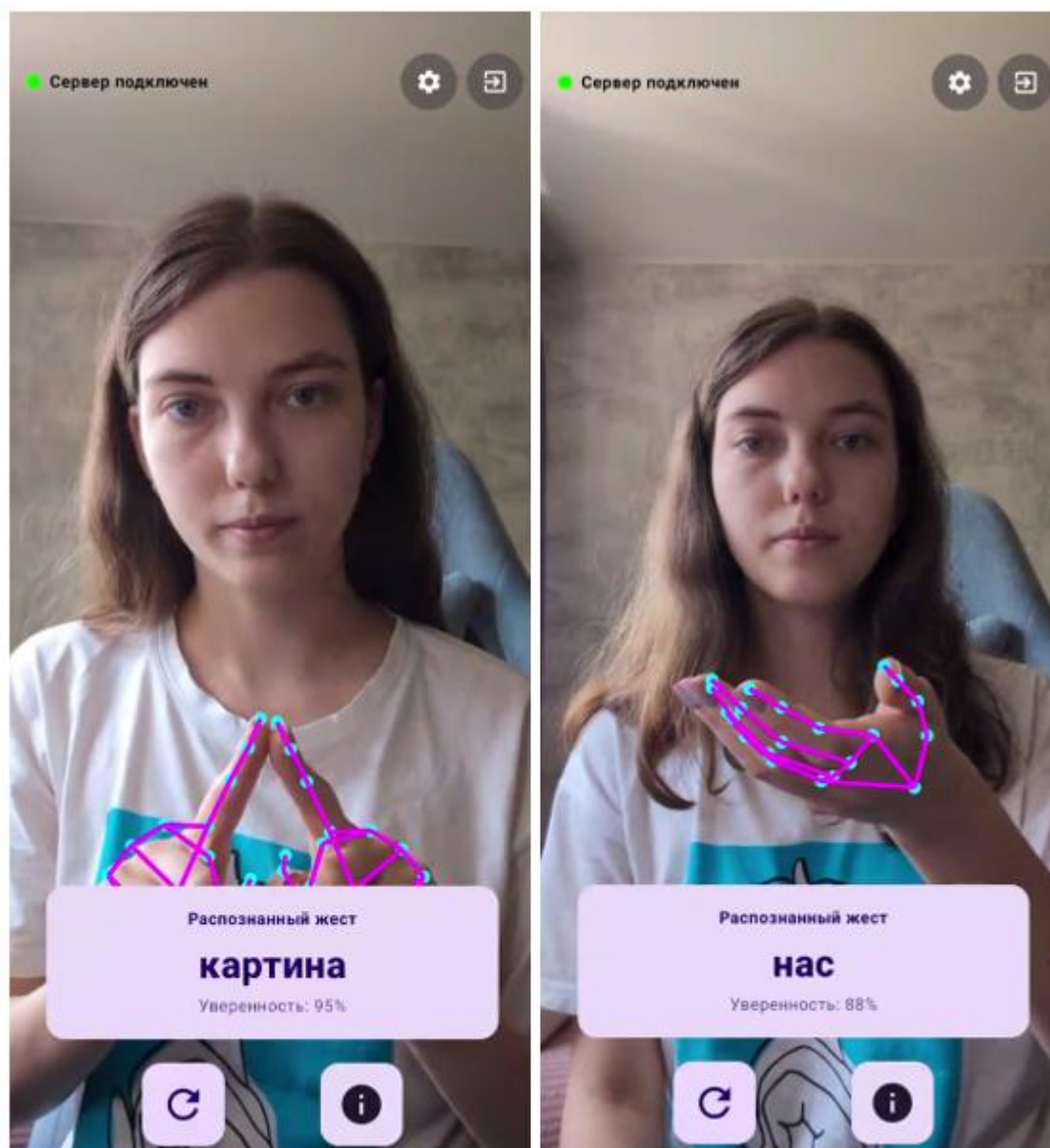


Рис.4.2. Отображение результатов распознавания

Система также обрабатывает различные состояния: отсутствие соединения с сервером, ошибки распознавания, низкий уровень уверенности модели. В этих

случаях карточка с распознанным словом не будет отображена на экране пользователя.

4.2. Разработка сервера

4.2.1. Прием и валидация данных от клиента

Точкой входа для данных о жестах служит API-эндпоинт `/features`, реализованный в функции `process_gesture`. Этот метод принимает HTTP POST-запросы с JSON-телом, содержащим массив признаков жеста и метаданные запроса, и возвращает результат распознавания в том же HTTP-ответе.

При получении запроса сервер выполняет многоуровневую валидацию входных данных. Сначала проверяется корректность Content-Type заголовка - запрос должен содержать `application/json`. Затем парсится JSON-тело и извлекается массив `features`, который должен содержать числовые значения координат ключевых точек рук. Код функции представлен в приложении 7 в листинге 7.

Система поддерживает полные последовательности жестов (2520 признаков для 20 кадров) для непосредственного распознавания. Для каждого формата применяются специфические алгоритмы валидации.

Ключевым аспектом валидации является проверка качества данных. Функция анализирует количество ненулевых значений в массиве признаков - если их доля составляет менее 10% от общего объема, такие данные считаются "пустыми" и отбраковываются. Это предотвращает попытки распознавания на кадрах, где руки отсутствуют или плохо видны.

После успешной валидации данные преобразуются в массивы NumPy с типом `float32`, немедленно передаются в модель машинного обучения для распознавания, и результат возвращается клиенту в JSON-формате в рамках того же HTTP-ответа. Одновременно фиксируется временная метка обработки данных сервером для последующего анализа производительности.

4.2.2. Подготовка данных для нейронной сети

Перед передачей в модель машинного обучения данные проходят финальную стадию подготовки в функции `recognize_gesture`. Этот процесс включает проверку структуры данных, нормализацию и преобразование в формат, ожидаемый нейронной сетью.

Система выполняет строгую валидацию входной последовательности: проверяется количество кадров (должно быть 20), размерность каждого кадра (126 признаков) и тип данных (массивы `NumPy`). Любое несоответствие приводит к возврату ошибки без попытки распознавания.

Дополнительно анализируется качество данных через подсчет ненулевых элементов. Если доля информативных данных составляет менее 3% от общего объема, последовательность считается некачественной. Этот порог снижен по сравнению с валидацией отдельных кадров для повышения чувствительности системы. Код функции представлен в приложении 8 в листинге 8.

4.2.3. Процесс распознавания с помощью нейронной сети

Загруженная модель применяется к подготовленным данным через метод `model.predict()`. Модель возвращает вектор вероятностей размерностью, соответствующей количеству классов жестов в обучающем датасете (30 классов в текущей реализации).

Каждый элемент выходного вектора представляет вероятность принадлежности входной последовательности к соответствующему классу жестов. Значения нормализованы и находятся в диапазоне от 0 до 1, где сумма всех вероятностей равна единице. Пример процесса распознавания показан на рис.4.3.

```

2025-06-09 19:44:46,399 - INFO - gesture_processor.py:recognize_gesture - Жест распознан: хороший (id: 26) | Уверенность: 0.866 | Top-5: хороший: 0.866, газета: 0.055, следующий: 0.021, ё: 0.017, вода: 0.009
2025-06-09 19:44:46,399 - INFO - gesture_routes.py:process_gesture - Распознан: хороший (0.866)
2025-06-09 19:44:46,399 - INFO - _internal.py:_log - 192.168.0.111 - - [09/Jun/2025 19:44:46] "POST /features HTTP/1.1" 200 -
2025-06-09 19:44:47,297 - INFO - gesture_processor.py:recognize_gesture - Жест распознан: я (id: 29) | Уверенность: 1.000 | Top-5: я: 1.000, одиннадцать: 0.000, довольно: 0.000, жд
ать: 0.000, переводить: 0.000
2025-06-09 19:44:47,297 - INFO - gesture_routes.py:process_gesture - Распознан: я (1.000)
2025-06-09 19:44:47,297 - INFO - _internal.py:_log - 192.168.0.111 - - [09/Jun/2025 19:44:47] "POST /features HTTP/1.1" 200 -
2025-06-09 19:44:47,667 - INFO - _internal.py:_log - 192.168.0.111 - - [09/Jun/2025 19:44:47] "GET / HTTP/1.1" 200 -
2025-06-09 19:44:49,969 - INFO - gesture_processor.py:recognize_gesture - Низкая уверенность (0.749 < 0.750). Наиболее вероятный класс: одиннадцать (id: 14), but result not returned.
2025-06-09 19:44:49,970 - INFO - _internal.py:_log - 192.168.0.111 - - [09/Jun/2025 19:44:49] "POST /features HTTP/1.1" 200 -
2025-06-09 19:44:52,651 - INFO - _internal.py:_log - 192.168.0.111 - - [09/Jun/2025 19:44:52] "GET / HTTP/1.1" 200 -
2025-06-09 19:44:57,654 - INFO - _internal.py:_log - 192.168.0.111 - - [09/Jun/2025 19:44:57] "GET / HTTP/1.1" 200 -
2025-06-09 19:44:57,818 - INFO - gesture_processor.py:recognize_gesture - Жест распознан: вода (id: 3) | Уверенность: 0.997 | Top-5: вода: 0.997, с: 0.002, ё: 0.001, способ: 0.000,
шестнадцать: 0.000

```

Рис.4.3. Процесс распознавания жестов

Система определяет предсказанный класс через функцию `np.argmax()`, которая находит индекс элемента с максимальным значением. Соответствующая вероятность извлекается как мера уверенности модели в своем предсказании.

Для повышения надежности результатов применяется пороговая фильтрация. Жест считается успешно распознанным только если уверенность модели превышает конфигурируемый порог (по умолчанию 0.75). Результаты с низкой уверенностью отбрасываются как ненадежные.

Результаты распознавания формируются непосредственно в функции `process_gesture` и немедленно возвращаются клиенту в JSON-формате в рамках того же HTTP-запроса. Это обеспечивает синхронную модель взаимодействия, при которой клиент получает результат сразу после отправки данных.

Функция `recognize_gesture` выполняет анализ последовательности признаков с помощью модели машинного обучения и возвращает структуру с названием жеста, уровнем уверенности и идентификатором класса. Эти данные дополняются серверной временной меткой и немедленно передаются клиенту, что позволяет оценить время обработки запроса и обеспечивает быстрый отклик системы.

4.3. Реализация и обучение нейронных сетей

Для решения задачи распознавания жестов русского жестового языка были реализованы и сравнены три различных архитектуры нейронных сетей: LSTM, GRU и 3D CNN. Каждая архитектура имеет свои особенности и преимущества при работе с временными последовательностями данных о движении рук.

Модель основана на двух последовательных двунаправленных LSTM-слоях (с 128 и 320 нейронами), что позволяет анализировать временные последовательности жестов в обоих направлениях для лучшего понимания контекста. После рекуррентных слоев применяется GlobalAveragePooling1D для агрегации данных со всей последовательности в единый вектор признаков. Завершает архитектуру блок из двух полносвязных (Dense) слоев на 256 и 128 нейронов с активацией ReLU.

Для борьбы с переобучением и стабилизации обучения по всей модели используются BatchNormalization после основных слоев, а также Dropout и L1/L2 регуляризация в полносвязной части. Архитектура LSTM-модели отражена на рис.4.4.

```

model = Sequential()

# 1. Первый слой Bidirectional LSTM
model.add(Bidirectional(LSTM(128, return_sequences=True,
                             activation='tanh',
                             recurrent_dropout=0.1),
                             input_shape=(frames, num_features)))
model.add(BatchNormalization())
model.add(Dropout(dropout_rate))

# 2. Второй слой Bidirectional LSTM
model.add(Bidirectional(LSTM(320, return_sequences=True,
                             activation='tanh',
                             recurrent_dropout=0.1)))
model.add(BatchNormalization())
model.add(Dropout(dropout_rate))

# 3. Глобальный пулинг для извлечения важной информации из последовательности
model.add(GlobalAveragePooling1D())
model.add(BatchNormalization())
model.add(Dropout(dropout_rate))

# 4. Полносвязные слои
model.add(Dense(256, kernel_regularizer=regularizers.l1_l2(l1=1e-6, l2=5e-4)))
model.add(Activation('relu'))
model.add(BatchNormalization())
model.add(Dropout(dropout_rate))

model.add(Dense(128, kernel_regularizer=regularizers.l1_l2(l1=1e-6, l2=5e-4)))
model.add(Activation('relu'))
model.add(BatchNormalization())
model.add(Dropout(dropout_rate/2))

# Выходной слой
model.add(Dense(num_classes, activation='softmax'))

```

Рис.4.4. Архитектура LSTM-модели

GRU-архитектура во многом аналогична LSTM-модели, но использует упрощенные рекуррентные ячейки. GRU объединяет входной и забывающий шлюзы LSTM в единый шлюз обновления, что приводит к меньшему количеству параметров и потенциально более быстрому обучению. Первый Bidirectional GRU-слой содержит 128 нейронов, а второй увеличивается до 320, сохраняя ту же прогрессию сложности, что и в LSTM-варианте. Ключевое различие между нейронными сетями заключается в механизме управления информационным потоком. GRU использует reset gate для управления тем, какая информация из прошлого состояния должна быть забыта, и update gate для определения того, какая

новая информация должна быть сохранена. Этот упрощенный механизм часто демонстрирует сопоставимую с LSTM производительность при меньших вычислительных затратах.

Остальная архитектура GRU-модели полностью повторяет LSTM-вариант, включая GlobalAveragePooling1D, полносвязные слои с регуляризацией и выходной softmax-слой. Архитектура GRU-модели отражена на рис.4.5.

```
# 1. Первый слой Bidirectional LSTM
model.add(Bidirectional(GRU(128, return_sequences=True,
                             activation='tanh',
                             recurrent_dropout=0.1),
                             input_shape=(frames, num_features)))
model.add(BatchNormalization())
model.add(Dropout(dropout_rate))

# 2. Второй слой Bidirectional LSTM
model.add(Bidirectional(GRU(320, return_sequences=True,
                             activation='tanh',
                             recurrent_dropout=0.1)))
model.add(BatchNormalization())
model.add(Dropout(dropout_rate))

# 3. Глобальный пулинг для извлечения важной информации из последовательности
model.add(GlobalAveragePooling1D())
model.add(BatchNormalization())
model.add(Dropout(dropout_rate))

# 4. Полносвязные слои
model.add(Dense(256, kernel_regularizer=regularizers.l1_l2(l1=1e-6, l2=5e-4)))
model.add(Activation('relu'))
model.add(BatchNormalization())
model.add(Dropout(dropout_rate))

model.add(Dense(128, kernel_regularizer=regularizers.l1_l2(l1=1e-6, l2=5e-4)))
model.add(Activation('relu'))
model.add(BatchNormalization())
model.add(Dropout(dropout_rate/2))

# Выходной слой
model.add(Dense(num_classes, activation='softmax'))
```

Рис.4.5. Архитектура GRU-модели

3D CNN-архитектура использует трехмерные сверточные слои для извлечения пространственных и временных признаков из видеоданных. В отличие от рекуррентных сетей, которые обрабатывают последовательности, 3D CNN

рассматривает видео как объем данных (высота, ширина, время) и применяет к нему трехмерные фильтры.

Модель начинается с трех последовательных блоков 3D-свертки. Первый сверточный слой имеет 32 фильтра, второй — 64, а третий — 128, что обеспечивает постепенное усложнение извлекаемых признаков. Каждый сверточный слой использует ядро размером (3, 3, 3) и функцию активации 'relu'. После каждой свертки применяются слой батч-нормализации для стабилизации обучения, слой MaxPooling3D для уменьшения пространственной размерности данных и Dropout для предотвращения переобучения.

После извлечения признаков сверточными слоями, полученные карты признаков выравниваются в один вектор с помощью слоя Flatten. Далее следуют два полносвязных слоя на 256 и 128 нейронов с relu-активацией, L2-регуляризацией и Dropout, которые выполняют классификацию на основе извлеченных признаков. Выходной слой использует softmax-активацию для получения вероятностного распределения по классам. Такая архитектура позволяет эффективно анализировать видеопотоки, одновременно улавливая как внешний вид объектов, так и их динамику во времени. Архитектура 3D CNN-модели отражена на рис.4.6.

```

model = Sequential()

# Первый блок 3D сверток с более глубокой структурой
model.add(Conv3D(32, kernel_size=(3, 3, 3), padding='same', activation='relu',
|   |   |   |   input_shape=(frames, height, width, channels)))
model.add(BatchNormalization())
model.add(Conv3D(32, kernel_size=(3, 3, 3), padding='same', activation='relu'))
model.add(BatchNormalization())
model.add(MaxPooling3D(pool_size=(2, 2, 2), padding='same'))
model.add(Dropout(0.2))

# Второй блок 3D сверток
model.add(Conv3D(64, kernel_size=(3, 3, 3), padding='same', activation='relu'))
model.add(BatchNormalization())
model.add(Conv3D(64, kernel_size=(3, 3, 3), padding='same', activation='relu'))
model.add(BatchNormalization())
model.add(MaxPooling3D(pool_size=(2, 2, 2), padding='same'))
model.add(Dropout(0.25))

# Третий блок 3D сверток
model.add(Conv3D(128, kernel_size=(3, 3, 3), padding='same', activation='relu'))
model.add(BatchNormalization())
model.add(Conv3D(128, kernel_size=(3, 3, 3), padding='same', activation='relu'))
model.add(BatchNormalization())
model.add(MaxPooling3D(pool_size=(2, 2, 2), padding='same'))
model.add(Dropout(0.3))

# Четвертый блок 3D сверток для более сложных паттернов
model.add(Conv3D(256, kernel_size=(3, 3, 3), padding='same', activation='relu'))
model.add(BatchNormalization())
model.add(MaxPooling3D(pool_size=(2, 2, 2), padding='same'))
model.add(Dropout(0.35))

# Глобальный усредненный пулинг вместо Flatten для уменьшения переобучения
model.add(GlobalAveragePooling3D())

# Улучшенные полносвязные слои с лучшей регуляризацией
model.add(Dense(512, kernel_regularizer=regularizers.l2(0.001)))
model.add(BatchNormalization())
model.add(Activation('relu'))
model.add(Dropout(dropout_rate))

model.add(Dense(256, kernel_regularizer=regularizers.l2(0.001)))
model.add(BatchNormalization())
model.add(Activation('relu'))
model.add(Dropout(dropout_rate/2))

model.add(Dense(128, kernel_regularizer=regularizers.l2(0.0005)))
model.add(BatchNormalization())
model.add(Activation('relu'))
model.add(Dropout(dropout_rate/3))

# Выходной слой
model.add(Dense(num_classes, activation='softmax'))

```

Рис.4.6. Архитектура 3D CNN-модели

Таким образом, в данной главе была создана полностью функционирующая система. Следующим логическим шагом является её тестирование для оценки точности, производительности и удобства, чему и будет посвящён следующий раздел.

ГЛАВА 5. ТЕСТИРОВАНИЕ МОБИЛЬНОГО ПРИЛОЖЕНИЯ

5.1. Тестирование системы

С целью оценки соответствия разработанной системы распознавания жестов РЖЯ заявленным требованиям был выполнен комплекс тестов. Методология тестирования, основанная на наборе сценариев (тест-кейсов), изложена в табл.5.1.

Таблица 5.1

Набор тест-кейсов

Действие	Результат
Запуск приложения без авторизации	Переход на стартовый экран с кнопками "Зарегистрироваться" и "Войти"
Регистрация нового пользователя	Создание учетной записи с валидными данными (имя, email, пароль). Автоматический переход на экран распознавания жестов после успешной регистрации
Вход в систему с корректными данными	Успешная авторизация и переход на основной экран с активацией камеры
Предоставление разрешения на использование камеры	Запуск видеопотока с камеры устройства, отображение ключевых точек рук при их обнаружении
Выполнение жеста из поддерживаемого словаря	Распознавание жеста с отображением результата в виде текста и уровня уверенности модели
Отсутствие рук в кадре	Корректная обработка пустых кадров без возникновения ошибок или сбоев приложения

Продолжение табл.5.1.

Переключение между фронтальной и задней камерой	Плавное переключение камеры без потери соединения с сервером и сохранением функциональности распознавания
Проверка работы с недоступным сервером	Отображение индикатора "Сервер недоступен", распознавание жестов не выполняется
Повторный запуск приложения авторизованным пользователем	Автоматический вход в систему без повторного ввода данных
Выход из системы	Удаление токена сессии и переход на стартовый экран
Вызов справки приложения	Отображение диалогового окна с инструкциями по использованию приложения

Проверка показала, что реализованный функционал работает корректно, так как в ходе каждого теста фактический результат совпадал с ожидаемым.

В результате ручного тестирования приложения отклонений в работе системы не обнаружено. Все элементы пользовательского интерфейса отображаются и работают корректно, данные о жестах передаются на сервер полностью, каждому распознанному жесту соответствует правильное текстовое представление.

Распознавание жестов осуществляется в режиме реального времени, обновление экрана с выводом результата происходит в течение 2–3 секунд после завершения выполнения жеста. Визуализация ключевых точек рук отображается корректно, позволяя пользователю контролировать качество захвата жеста.

Система аутентификации функционирует без сбоев. Автоматический вход в систему при повторном запуске приложения работает стабильно.

5.2. Совместимость с устройствами

Приложение было протестировано на различных устройствах с разными версиями Android и характеристиками экрана. Используемые для тестирования устройства отображены в табл.5.2.

Таблица 5.2

Устройства для тестирования приложения

Устройство	ОС	Количество мега- пикселей фронтальной камеры	Расширение экрана
Pixel 8 Pro	Android 16	10.5 Мп	2992x1344
POCO F5	Android 13	16 Мп	2400x1080
TECNO CAMON 19 Pro	Android 13	32 Мп	2460x1080

Проблем в верстке не было обнаружено при тестировании на устройствах с указанными разрешениями экрана. На всех указанных телефонах система распознавания работает корректно.

5.3. Сравнение нейронных сетей

Для объективной оценки эффективности различных архитектур нейронных сетей в задаче распознавания жестов русского жестового языка был проведен комплексный анализ их производительности. Сравнение включало оценку точности распознавания на тестовой выборке и времени обучения.

Все модели обучались на одинаковом наборе данных из датасета SLOVO, содержащем 30 классов жестов. Тренировочная выборка составила 80% от общего объема данных, валидационная и тестовая по 10% каждая. Для обеспечения объективности сравнения использовались идентичные гиперпараметры обучения везде, где это было возможно, включая batch size, функцию потерь, метрики и стратегию регуляризации.

LSTM-модель продемонстрировала наилучшую производительность среди всех тестируемых архитектур, достигнув точности 98.53% на тестовой выборке. Время обучения заняло 2 минуты и 52 секунды. На рис.5.1 показано, что нейронная сеть смогла распознать практически все слова.

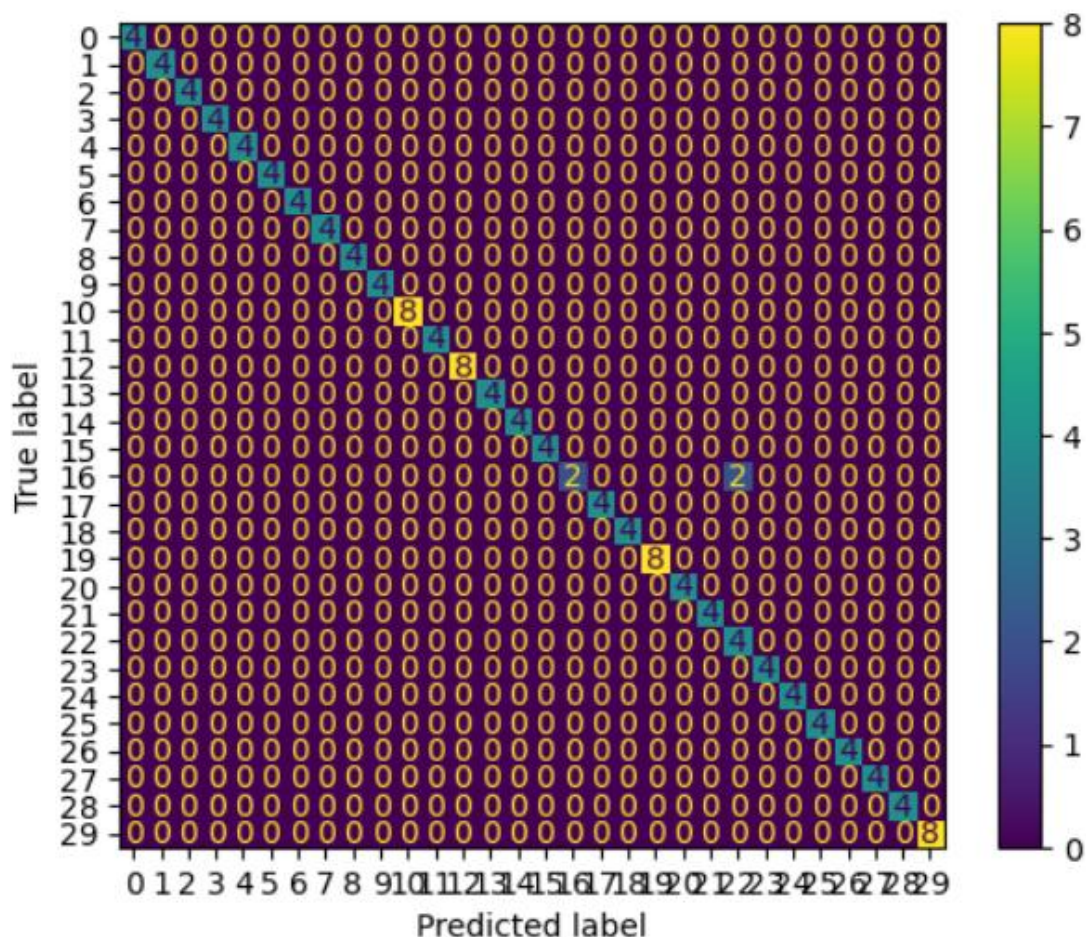


Рис.5.1. LSTM ConfusionMatrix

GRU-архитектура показала результат 83.82% точности при существенно меньшем количестве параметров, чем у LSTM. Обучилась модель за 1 минуту 49 секунд. Данная нейронная сеть уступает в качестве LSTM, это видно на рис.5.2.

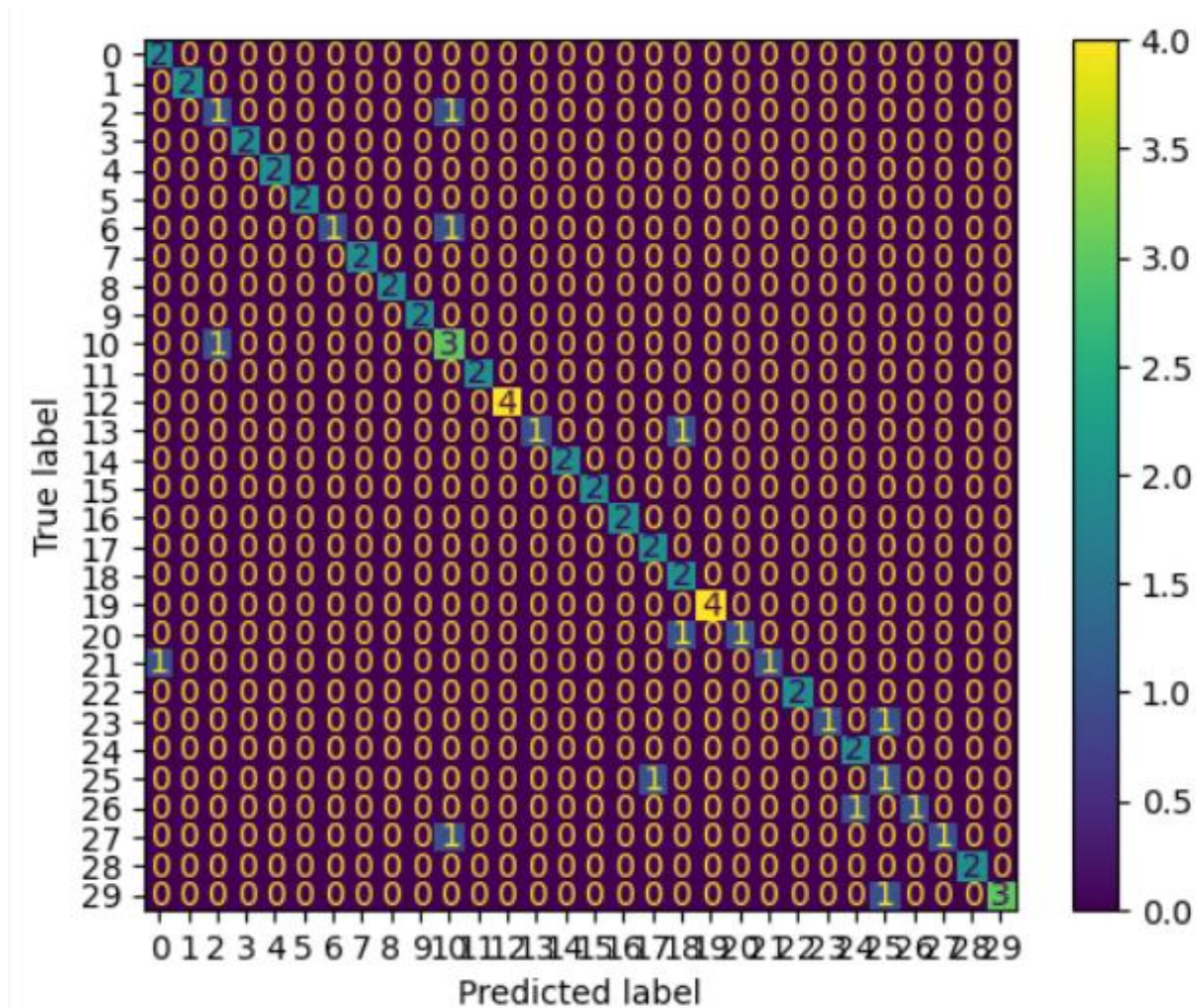


Рис.5.2. GRU ConfusionMatrix

3D CNN-модель достигла точности 79.41% на тестовой выборке. Время обучения заняло 2 минуты 48 секунд. Несмотря на неплохой результат на тестовой выборке, данная нейронная сеть значительно больше ошибалась в распознавании, чем GRU. Некоторые жесты она так и не смогла распознать, что показано на рис.5.3.

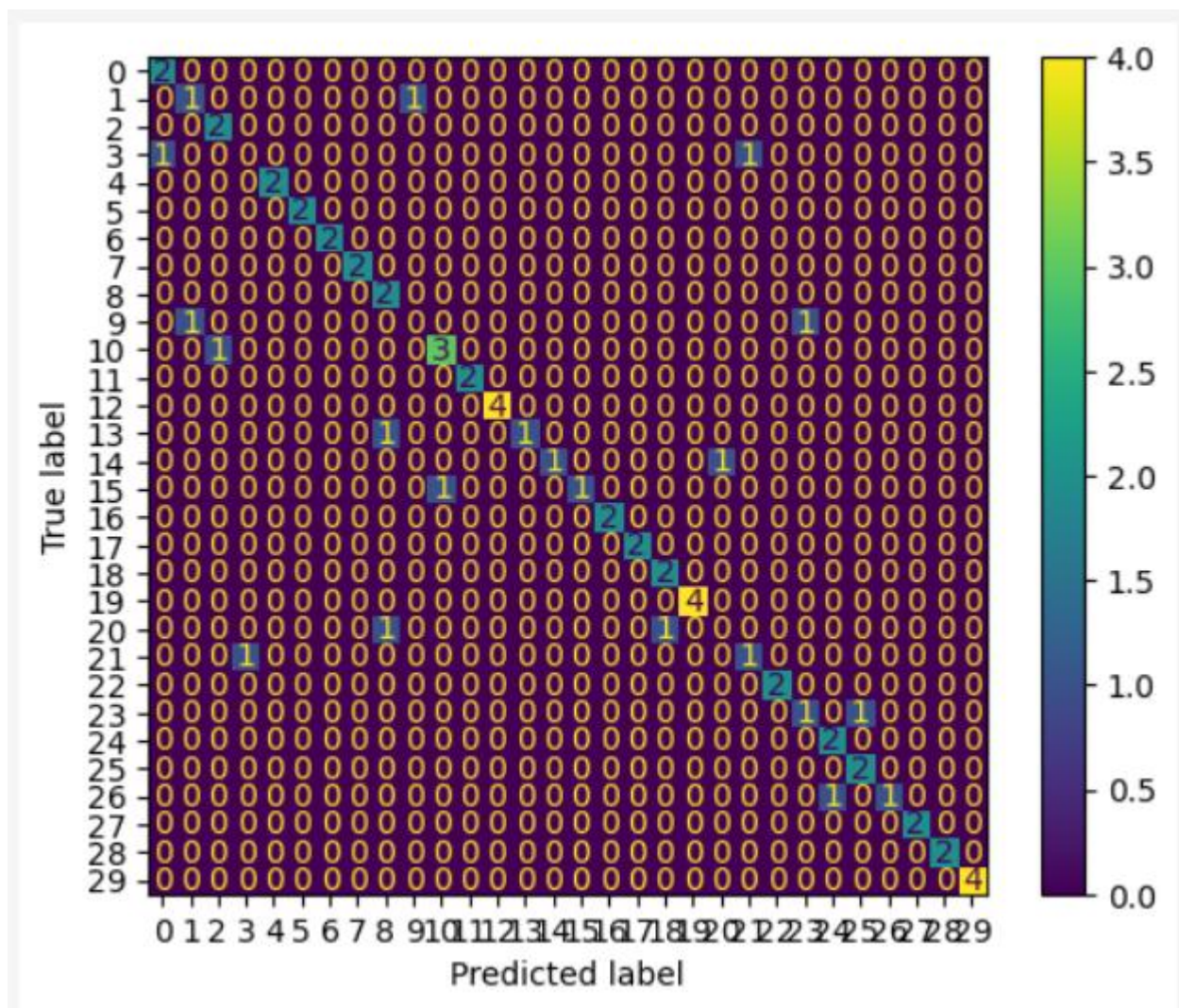


Рис.5.3. 3D CNN ConfusionMatrix

Таким образом, наилучшим вариантом оказалась LSTM, именно поэтому она является нейронной сетью по умолчанию в данном проекте.

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы была успешно разработана система распознавания жестов русского жестового языка, состоящая из мобильного приложения для платформы Android и серверной части для обработки и распознавания жестов.

В процессе выполнения работы были достигнуты следующие результаты:

- 1) Проведен анализ особенностей русского жестового языка как объекта автоматического распознавания, изучены существующие решения в данной области. Установлено отсутствие аналогичных мобильных приложений для РЖЯ на российском рынке.
- 2) Разработана клиент-серверная архитектура системы, обеспечивающая эффективное распределение вычислительной нагрузки между мобильным устройством и сервером. Такой подход позволил снизить требования к ресурсам мобильного устройства и обеспечить возможность централизованного обновления моделей машинного обучения.
- 3) Реализовано мобильное приложение с использованием современного стека технологий: Kotlin, Jetpack Compose, MediaPipe Hand Landmarker. Приложение обеспечивает захват видеопотока с камеры, детекцию ключевых точек рук, визуализацию результатов и взаимодействие с сервером.
- 4) Разработан интеллектуальный анализатор жестов GesturePhaseDetector, способный определять фазы выполнения жеста и формировать оптимальные последовательности кадров для распознавания.
- 5) Реализованы и обучены три различные архитектуры нейронных сетей: LSTM, GRU и 3D CNN. Проведенное сравнительное тестирование показало, что LSTM-модель демонстрирует наилучшую производительность с точностью распознавания 98.53% на тестовой выборке.

- 6) Система успешно протестирована на различных мобильных устройствах и показала стабильную работу с временем отклика 2–3 секунды от завершения жеста до отображения результата.

Разработанное решение имеет важное социальное значение, так как способствует преодолению коммуникационного барьера между слышащими и неслышащими людьми. Мобильный формат приложения обеспечивает доступность системы в любое время и в любом месте, что особенно важно для повседневного общения.

Данная работа была представлена на Неделе науки, где получила положительную оценку экспертов за практическую значимость и инновационный подход к решению проблемы коммуникации с носителями русского жестового языка.

Перспективы развития проекта включают:

- Расширение словаря распознаваемых жестов до полного объема датасета SLOVO (1000 жестов)
- Добавление функции распознавания целых фраз и предложений
- Разработка версии приложения для iOS
- Интеграция синтеза речи для озвучивания распознанных жестов
- Создание обратного функционала - перевода речи в жестовый язык

Таким образом, все поставленные задачи выполнены в полном объеме, цель работы достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Борисова Л. В. Пространство в жестовых языках как лингвофилософская категория [Электронный ресурс] // КиберЛенинка. – URL: <https://cyberleninka.ru/article/n/prostranstvo-v-zhestovyh-yazykah-kak-lingvofilosofskaya-kategoriya> (дата обращения 02.03.2025) [9]
2. Глухота и потеря слуха [Электронный ресурс] // Всемирная организация здравоохранения. – URL: www.who.int/ru/news-room/fact-sheets/detail/deafness-and-hearing-loss (дата обращения 14.02.2025).
3. Зайцева Г. Л. Жестовая речь. Дактилология: Учеб. для студ. высш. учебных заведений. – М.: Гуманит. изд. центр ВЛАДОС, 2000. — 192 с.
4. Инвалидов по зрению и слуху в России в 200 раз больше, чем по официальной статистике [Электронный ресурс] // GxP news. – URL: <https://gxpnews.net/2022/11/invalidov-po-zreniyu-i-sluhu-v-rossii-v-200-raz-bolshe-chem-po-oficzialnoj-statistike/> (дата обращения 14.02.2025).
5. Иностранные языки жестов. Отличаются ли они в разных странах? [Электронный ресурс] // Unipage. – URL: https://www.unipage.net/ru/sign_language (дата обращения 14.02.2025).
6. Исследование: в России пользователей смартфонов на Android в 1,5 раза больше, чем iPhone [Электронный ресурс] // Telecom Daily. – URL: <https://telecomdaily.ru/news/2024/12/09/issledovanie-v-rossii-polzovateley-smartfonov-na-android-v-1-5-raza-bolshe-chem-iphone> (дата обращения: 20.02.2025).
7. Итоги ВПН-2020. Том-5 Национальный состав и владение языками. Таблица 4 // Федеральная служба государственной статистики (дата обращения 14.02.2025).
8. Международный день жестовых языков 23 сентября [Электронный ресурс] // Организация Объединенных Наций. – URL: <https://www.un.org/ru/observances/sign-languages-day> (дата обращения 13.02.2025).

9. Расширенная грамматика РЖЯ [Электронный ресурс] // Языки меньшинств России. Институт языкознания РАН. – URL: <https://minlang.iling-ran.ru/grammar/rasshirennaya-grammatika-rzhyu> (дата обращения 11.03.2025).
10. Федеральный закон "О внесении изменений в статьи 14 и 19 Федерального закона "О социальной защите инвалидов в Российской Федерации" от 30.12.2012 N 296-ФЗ (последняя редакция) // Собрание законодательства РФ (дата обращения 14.02.2025).
11. 3D Convolutional Neural Network — A Guide for Engineers [Электронный ресурс] // Neural Concept. – URL: <https://www.neuralconcept.com/post/3d-convolutional-neural-network-a-guide-for-engineers> (дата обращения 21.02.2025).
12. Abadi M., Barham P., Chen J. TensorFlow: A system for large-scale machine learning [Электронный ресурс] // arXiv preprint. – URL: <https://arxiv.org/pdf/1605.08695>
13. Cao Z., Simon T. OpenPose: Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields // arXiv preprint. – URL: <https://arxiv.org/pdf/1812.08008> (дата обращения 15.03.2025).
14. Gated Recurrent Unit Networks [Электронный ресурс] // GeeksforGeeks. – URL: <https://www.geeksforgeeks.org/gated-recurrent-unit-networks/>
15. Hand landmarks detection guide [Электронный ресурс] // Google AI for Developers. – URL: https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker
16. Jetpack Compose Documentation [Электронный ресурс] // Google for Android Developers. – URL: <https://developer.android.com/develop/ui/compose/documentation> (дата обращения 21.02.2025).
17. Kapitanov A., Kvanchiani K., Nagaev A., Petrova E. Slovo: Russian Sign Language Dataset [Электронный ресурс] // arXiv preprint. – URL: <https://arxiv.org/pdf/2305.14527>
18. MediaPipe Hands [Электронный ресурс] // GitHub. – URL: <https://clck.ru/3MYPNY>

19. Pulli K., Baksheev A., Korniyakov K., Eruhimov V. Real-Time Computer Vision with OpenCV [Электронный ресурс] // Nvidia. URL: https://research.nvidia.com/sites/default/files/pubs/2012-06_Realtime-Computer-Vision/OpenCV_CACM_p61-pulli.pdf
20. TensorFlow Lite Gesture Classification [Электронный ресурс] // GitHub. – URL: https://github.com/tensorflow/examples/tree/master/lite/examples/gesture_classification/android
21. What is LSTM - Long Short-Term Memory? [Электронный ресурс] // Geeks-forGeeks. – URL: <https://www.geeksforgeeks.org/deep-learning-introduction-to-long-short-term-memory/>

Приложение 1. CameraHelper

Класс `CameraHelper` автоматически выбирает оптимальную камеру устройства, обеспечивает переключение между ними через метод `toggleCamera` и управляет ресурсами камеры с помощью собственного потока `cameraExecutor`, избегая блокировки UI. Жизненный цикл камеры контролируется, а ресурсы освобождаются при выходе из приложения или переходе в фон. Для анализа используется `ImageAnalyzer`, обрабатывающий только свежие кадры.

Листинг 1. Код класса `CameraHelper`

```
/**
 * Класс для настройки камеры и анализа изображений
 */
class CameraHelper(
    private val context: Context
) {
    private var camera: androidx.camera.core.Camera? = null
    private var preview: Preview? = null
    private var imageAnalysis: ImageAnalysis? = null
    private var imageCapture: ImageCapture? = null
    private var cameraExecutor: ExecutorService = Executors.newSingleThreadExecutor()
    private var cameraProvider: ProcessCameraProvider? = null
    private var currentCameraSelector = CameraSelector.LENS_FACING_BACK
    private val processingImage = AtomicBoolean(false)
    private val mainHandler = Handler(Looper.getMainLooper())
    private val isSwitchingCamera = AtomicBoolean(false)
    private var frameListener: ((ImageProxy) -> Unit)? = null
    init {
        val cameraManager = context.getSystemService(Context.CAMERA_SERVICE) as CameraManager
        val cameraIdList = cameraManager.cameraIdList
        if (cameraIdList.isEmpty()) {
            //
        } else {
            var hasFrontCamera = false
            var hasBackCamera = false

            for (cameraId in cameraIdList) {
                val characteristics = cameraManager.getCameraCharacteristics(cameraId)
                val facing = characteristics.get(CameraCharacteristics.LENS_FACING)
                if (facing == CameraCharacteristics.LENS_FACING_FRONT) {
                    hasFrontCamera = true
                } else if (facing == CameraCharacteristics.LENS_FACING_BACK) {
                    hasBackCamera = true
                }
            }
            currentCameraSelector = when {
                hasFrontCamera -> CameraSelector.LENS_FACING_FRONT
                hasBackCamera -> CameraSelector.LENS_FACING_BACK
                else -> {
                    CameraSelector.LENS_FACING_BACK
                }
            }
        }
    }
}
```

```

    }
    }
}

/**
 * Запуск камеры
 */
@OptIn(ExperimentalGetImage::class)
suspend fun startCamera(
    lifecycleOwner: LifecycleOwner,
    surfaceProvider: Preview.SurfaceProvider,
    cameraSelector: Int = currentCameraSelector
): Boolean = suspendCancellableCoroutine { continuation ->
    try {
        currentCameraSelector = cameraSelector
        val cameraProviderFuture = ProcessCameraProvider.getInstance(context)
        cameraProviderFuture.addListener({
            try {
                val provider = cameraProviderFuture.get()
                cameraProvider = provider
                provider.unbindAll()
                preview = Preview.Builder()
                    .build()
                    .also {
                        it.surfaceProvider = surfaceProvider
                    }
                imageCapture = ImageCapture.Builder()
                    .build()
                imageAnalysis = ImageAnalysis.Builder()
                    .setBackpressureStrategy(ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST)
                    .build()
                    .also {
                        it.setAnalyzer(cameraExecutor, ImageAnalyzer())
                    }
                val selector = CameraSelector.Builder().requireLensFacing(cameraSelector).build()
                try {
                    camera = provider.bindToLifecycle(
                        lifecycleOwner,
                        selector,
                        preview,
                        imageCapture,
                        imageAnalysis
                    )
                    if (continuation.isActive) {
                        continuation.resume(true)
                    }
                } catch (e: Exception) {
                    e.printStackTrace()
                    if (continuation.isActive) {
                        continuation.resume(false)
                    }
                }
            } catch (e: Exception) {
                e.printStackTrace()

                if (continuation.isActive) {
                    continuation.resume(false)
                }
            }
        })
    }
}

```

```

    }
    }, ContextCompat.getMainExecutor(context))
} catch (e: Exception) {
    e.printStackTrace()
    if (continuation.isActive) {
        continuation.resume(false)
    }
}
}
}
/**
 * Переключение камеры между фронтальной и задней с очисткой
 */
suspend fun toggleCamera(
    lifecycleOwner: LifecycleOwner,
    surfaceProvider: Preview.SurfaceProvider
): Boolean {
    if (isSwitchingCamera.getAndSet(true)) {
        return false
    }
    try {
        shutdownCamera()
        delay(500)
        val newCameraSelector = if (currentCameraSelector == CameraSelector.LENS_FACING_FRONT) {
            CameraSelector.LENS_FACING_BACK
        } else {
            CameraSelector.LENS_FACING_FRONT
        }
        val result = startCamera(lifecycleOwner, surfaceProvider, newCameraSelector)

        return result
    } catch (e: Exception) {
        e.printStackTrace()
        return false
    } finally {
        isSwitchingCamera.set(false)
    }
}
/**
 * Отключение всех ресурсов камеры
 */
private suspend fun shutdownCamera() = withContext(Dispatchers.Main) {
    try {
        frameListener = null
        mainHandler.removeCallbacksAndMessages(null)
        processingImage.set(false)
        cameraProvider?.unbindAll()
        camera = null
        preview = null
        imageCapture = null
        imageAnalysis = null
    } catch (e: Exception) {
        e.printStackTrace()
    }
}
/**
 * Прослушиватель для обработки кадров
 */

```

```

fun setFrameListener(listener: (ImageProxy) -> Unit) {
    frameListener = listener
}
/**
 * Остановка камеры и сброс ресурсов
 */
fun shutdown() {
    try {
        mainHandler.removeCallbacksAndMessages(null)
        cameraProvider?.unbindAll()
        cameraExecutor.shutdown()
        try {
            if (!cameraExecutor.awaitTermination(1000, TimeUnit.MILLISECONDS)) {
                cameraExecutor.shutdownNow()
            }
        } catch (e: InterruptedException) {
            cameraExecutor.shutdownNow()
        }
        camera = null
        preview = null
        imageCapture = null
        imageAnalysis = null
        cameraProvider = null
        frameListener = null
    } catch (e: Exception) {
        e.printStackTrace()
    }
}
/**
 * Получить текущее направление камеры
 */
fun getCurrentCameraFacing(): Int {
    return currentCameraSelector
}
@ExperimentalGetImage
private inner class ImageAnalyzer : ImageAnalysis.Analyzer {
    override fun analyze(imageProxy: ImageProxy) {
        if (processingImage.getAndSet(true)) {
            imageProxy.close()
            return
        }
        try {
            frameListener?.invoke(imageProxy) ?: imageProxy.close()
        } catch (e: Exception) {
            try {
                imageProxy.close()
            } catch (e2: Exception) {
                Log.e(TAG, "Ошибка при закрытии imageProxy: ${e2.message}")
            }
        } finally {
            processingImage.set(false)
        }
    }
}
companion object {
    private const val TAG = "CameraHelper"
}
}

```

Приложение 2. HandLandmarkHelper

Класс HandLandmarkHelper выполняет распознавание положения до двух рук в реальном времени. Он получает кадры от CameraHelper, преобразует их в формат MediaPipe с учетом типа камеры, и возвращает результаты через LandmarkListener. Для каждой руки система предоставляет массив из 21 нормализованной 3D-точки. Это обеспечивает стабильную работу независимо от разрешения камеры. Ошибки распознавания обрабатываются без прерывания работы.

Листинг 2. Код класса HandLandmarkHelper

```
/**
 * Класс для определения ключевых точек рук с использованием MediaPipe Hand Landmark
 */
class HandLandmarkHelper(
    var minHandDetectionConfidence: Float = DEFAULT_HAND_DETECTION_CONFIDENCE,
    var minHandTrackingConfidence: Float = DEFAULT_HAND_TRACKING_CONFIDENCE,
    var minHandPresenceConfidence: Float = DEFAULT_HAND_PRESENCE_CONFIDENCE,
    var maxNumHands: Int = DEFAULT_NUM_HANDS,
    var currentDelegate: Int = DELEGATE_CPU,
    var runningMode: RunningMode = RunningMode.IMAGE,
    val context: Context,
    val handLandmarkHelperListener: LandmarkListener? = null
) {
    private var handLandmark: HandLandmarker? = null
    init {
        setupHandLandmark()
    }
    fun clearHandLandmark() {
        handLandmark?.close()
        handLandmark = null
    }
    private fun setupHandLandmark() {
        val baseOptionBuilder = BaseOptions.builder()
        when (currentDelegate) {
            DELEGATE_CPU -> {
                baseOptionBuilder.setDelegate(Delegate.CPU)
            }
            DELEGATE_GPU -> {
                baseOptionBuilder.setDelegate(Delegate.GPU)
            }
        }
        baseOptionBuilder.setModelAssetPath(MP_HAND_LANDMARK_TASK)
        when (runningMode) {
            RunningMode.LIVE_STREAM -> {
                if (handLandmarkHelperListener == null) {
                    throw IllegalStateException(
                        "Должен быть установлен handLandmarkHelperListener, когда runningMode = LIVE_STREAM."
                    )
                }
            }
        }
    }
}
```



```

    }
  }
  else -> {
    //
  }
}
try {
  val baseOptions = baseOptionBuilder.build()
  val optionsBuilder =
    HandLandmarker.HandLandmarkerOptions.builder()
      .setBaseOptions(baseOptions)
      .setMinHandDetectionConfidence(minHandDetectionConfidence)
      .setMinTrackingConfidence(minHandTrackingConfidence)
      .setMinHandPresenceConfidence(minHandPresenceConfidence)
      .setNumHands(maxNumHands)
      .setRunningMode(runningMode)

  if (runningMode == RunningMode.LIVE_STREAM) {
    optionsBuilder
      .setResultListener(this::returnLivestreamResult)
      .setErrorListener(this::returnLivestreamError)
  }
  val options = optionsBuilder.build()
  handLandmark =
    HandLandmarker.createFromOptions(context, options)
} catch (e: IllegalStateException) {
  handLandmarkHelperListener?.onError(
    "Не удалось инициализировать Hand Landmark. Смотри журналы ошибок для " +
    "подробностей"
  )
} catch (e: RuntimeException) {
  handLandmarkHelperListener?.onError(
    "Не удалось инициализировать Hand Landmark. Смотри журналы ошибок для " +
    "подробностей", GPU_ERROR
  )
}
}

/**
 * Преобразует ImageProxy в MpiImage и передает его в HandLandmark
 */
@OptIn(ExperimentalGetImage::class)
fun detectLiveStream(
  imageProxy: ImageProxy,
  isFrontCamera: Boolean
) {
  if (runningMode != RunningMode.LIVE_STREAM) {
    throw IllegalArgumentException(
      "Попытка вызвать detectLiveStream" +
      " при использовании режима, отличного от RunningMode.LIVE_STREAM"
    )
  }
  val frameTime = SystemClock.uptimeMillis()
  try {
    val mediaImage = imageProxy.image
    if (mediaImage == null) {
      imageProxy.close()
    }
    return
  }
}

```

```

    }

    // Размеры изображения
    val width = mediaImage.width
    val height = mediaImage.height

    // Данные изображения (YUV формат)
    val yBuffer = mediaImage.planes[0].buffer // Y
    val uBuffer = mediaImage.planes[1].buffer // U
    val vBuffer = mediaImage.planes[2].buffer // V

    // Размеры данных
    val ySize = yBuffer.remaining()
    val uSize = uBuffer.remaining()
    val vSize = vBuffer.remaining()

    // Массив для хранения YUV данных
    val nv21 = ByteArray(ySize + uSize + vSize)

    yBuffer.get(nv21, 0, ySize)
    vBuffer.get(nv21, ySize, vSize)
    uBuffer.get(nv21, ySize + vSize, uSize)

    // Для конвертации в JPEG
    val yuvImage = android.graphics.YuvImage(
        nv21,
        android.graphics.ImageFormat.NV21,
        width,
        height,
        null
    )

    // Конвертация YUV в JPEG
    val out = java.io.ByteArrayOutputStream()
    yuvImage.compressToJpeg(
        android.graphics.Rect(0, 0, width, height),
        100,
        out
    )

    // JPEG в Bitmap
    val jpegBytes = out.toByteArray()
    val bitmap = android.graphics.BitmapFactory.decodeByteArray(jpegBytes, 0, jpegBytes.size)

    imageProxy.close()

    // Ротация и отражение
    val matrix = android.graphics.Matrix().apply {
        postRotate(imageProxy.imageInfo.rotationDegrees.toFloat())
        if (isFrontCamera) {
            postScale(
                -1f,
                1f,
                bitmap.width.toFloat(),
                bitmap.height.toFloat()
            )
        }
    }
}

```

```

        val rotatedBitmap = Bitmap.createBitmap(
            bitmap,
            0,
            0,
            bitmap.width,
            bitmap.height,
            matrix,
            true
        )
        if (rotatedBitmap != bitmap) {
            bitmap.recycle()
        }
        // Конвертация в формат MediaPipe
        val mpImage = BitmapImageBuilder(rotatedBitmap).build()

        detectAsync(mpImage, frameTime)
    } catch (e: Exception) {
        e.printStackTrace()
        try {
            imageProxy.close()
        } catch (e2: Exception) {
            //
        }
    }
}

/**
 * Определение ключевых точек рук с использованием MediaPipe Hand Landmark API в асинхронном
режиме
 */
@VisibleForTesting
fun detectAsync(mpImage: MPIOImage, frameTime: Long) {
    handLandmark?.detectAsync(mpImage, frameTime)
}

/**
 * Возвращает результат определения ключевых точек
 */
private fun returnLivestreamResult(
    result: HandLandmarkerResult,
    input: MPIOImage
) {
    val finishTimeMs = SystemClock.uptimeMillis()
    val inferenceTime = finishTimeMs - result.timestampMs()
    handLandmarkHelperListener?.onResults(
        ResultBundle(
            listOf(result),
            inferenceTime,
            input.height,
            input.width
        )
    )
}

/**
 * Возвращает ошибки, возникшие во время обнаружения
 */
private fun returnLivestreamError(error: RuntimeException) {

```

```

        handLandmarkHelperListener?.onError(
            error.message ?: "Неизвестная ошибка"
        )
    }
    companion object {
        const val TAG = "HandLandmarkHelper"
        private const val MP_HAND_LANDMARK_TASK = "hand_landmarker.task"

        const val DELEGATE_CPU = 0
        const val DELEGATE_GPU = 1
        const val DEFAULT_HAND_DETECTION_CONFIDENCE = 0.5F
        const val DEFAULT_HAND_TRACKING_CONFIDENCE = 0.5F
        const val DEFAULT_HAND_PRESENCE_CONFIDENCE = 0.5F
        const val DEFAULT_NUM_HANDS = 1
        const val OTHER_ERROR = 0
        const val GPU_ERROR = 1
    }

    /**
     * Класс для хранения результатов определения ключевых точек рук
     */
    data class ResultBundle(
        val results: List<HandLandmarkerResult>,
        val inferenceTime: Long,
        val inputImageHeight: Int,
        val inputImageWidth: Int,
    )

    /**
     * Интерфейс для обратного вызова с результатами определения ключевых точек
     */
    interface LandmarkListener {
        fun onError(error: String, errorCode: Int = OTHER_ERROR)
        fun onResults(resultBundle: ResultBundle)
    }
}

```

Приложение 3. extractFeaturesFromLandmarks

Функция `extractFeaturesFromLandmarks` обрабатывает координаты ключевых точек, полученные от MediaPipe, перед отправкой на сервер. Она нормализует координаты для фронтальной камеры, отражая их по оси X, добавляет минимальный случайный шум для вариативности данных, и формирует вектор признаков размером 126 элементов (63 на руку). При отсутствии руки соответствующие позиции заполняются нулями с шумом.

Листинг 3. Код функции `extractFeaturesFromLandmarks`

```
/**
 * Преобразует внутренний формат MediaPipe в формат для API сервера с нормализацией.
 */
private fun extractFeaturesFromLandmarks(
    handLandmarkResult: HandLandmarkerResult,
    isFrontCamera: Boolean = false
): List<Float> {
    val features = mutableListOf<Float>()
    fun addRandomNoise(value: Float): Float {
        return value + (Math.random() * 0.0001 - 0.00005).toFloat()
    }
    var firstHandFeatures = emptyList<Float>()
    if (handLandmarkResult.landmarks().isNotEmpty()) {
        val landmarks = handLandmarkResult.landmarks()[0]
        if (landmarks.size == 21) {
            firstHandFeatures = landmarks.flatMap { landmark ->
                val x = if (isFrontCamera) 1.0f - landmark.x() else landmark.x()
                listOf(
                    addRandomNoise(x),
                    addRandomNoise(landmark.y()),
                    addRandomNoise(landmark.z())
                )
            }
        }
    }
    if (firstHandFeatures.isNotEmpty()) {
        features.addAll(firstHandFeatures)
    } else {
        features.addAll(List(63) { addRandomNoise(0f) })
    }
    var secondHandFeatures = emptyList<Float>()
    if (handLandmarkResult.landmarks().size > 1) {
        val landmarks = handLandmarkResult.landmarks()[1]
        if (landmarks.size == 21) {
            secondHandFeatures = landmarks.flatMap { landmark ->
                val x = if (isFrontCamera) 1.0f - landmark.x() else landmark.x()
                listOf(
                    addRandomNoise(x),
                    addRandomNoise(landmark.y()),
                    addRandomNoise(landmark.z())
                )
            }
        }
    }
}
```

```
    )  
    }  
  }  
}  
if (secondHandFeatures.isNotEmpty()) {  
    features.addAll(secondHandFeatures)  
} else {  
    features.addAll(List(63) { addRandomNoise(0f) })  
}  
return features  
}
```

Приложение 4. GesturePhaseDetector

Ключевым элементом обработки жестов является GesturePhaseDetector. Анализатор определяет фазы выполнения жеста: инициализацию, активную фазу, стабилизацию и завершение. Он отслеживает последовательность из 20 кадров, анализируя движение между ними. При завершении жеста GesturePhaseDetector выбирает наиболее информативные кадры и формирует оптимальный набор данных для передачи на сервер.

Листинг 4. Код класса GesturePhaseDetector

```
/**
 * Упрощенный детектор для определения фаз жеста
 */
object GesturePhaseDetector {
    enum class GesturePhase {
        NONE,          // Нет активного жеста
        INITIALIZING,  // Рука появилась и начинает движение
        MOVEMENT,       // Активная фаза движения жеста
        STABILIZING,    // Замедление движения, переход к завершению
        FINALIZING,     // Финальная поза жеста
        COMPLETED      // Жест полностью завершен
    }

    private const val MOVEMENT_THRESHOLD = 0.02f
    private const val STABILIZATION_THRESHOLD = 0.01f
    private const val STABILIZATION_FRAMES = 5
    private const val MIN_GESTURE_FRAMES = 15
    private const val MAX_HISTORY_SIZE = 50
    private const val TARGET_FRAMES = 20
    private var currentPhase = GesturePhase.NONE
    private var stabilizationCounter = 0
    private var totalFramesInGesture = 0
    private var movementHistoryBuffer = mutableListOf<MovementFrame>()
    private data class MovementFrame(
        val features: List<Float>,
        val timestamp: Long,
        val movementMagnitude: Float = 0f
    )
}

/**
 * Обрабатывает новый кадр движения рук
 */
fun processFrame(features: List<Float>, timestamp: Long): Pair<Boolean, List<List<Float>>> {
    val isEmpty = isEmptyFrame(features)
    if (isEmpty && currentPhase != GesturePhase.NONE) {
        val result = finalizeGesture()
        resetState()
        return result
    }
    if (isEmpty) return Pair(false, emptyList())
    val movementMagnitude = if (movementHistoryBuffer.isNotEmpty()) {
        calculateMovement(movementHistoryBuffer.last().features, features)
    }
```

```

    } else 0f
    movementHistoryBuffer.add(MovementFrame(features.toList(), timestamp, movementMagnitude))
    totalFramesInGesture++
    if (movementHistoryBuffer.size > MAX_HISTORY_SIZE) {
        movementHistoryBuffer.removeAt(0)
    }
    updateGesturePhase(movementMagnitude)
    if (currentPhase == GesturePhase.COMPLETED) {
        val result = finalizeGesture()
        resetState()
        return result
    }
    return Pair(false, emptyList())
}
/**
 * Обновляет текущую фазу жеста
 */
private fun updateGesturePhase(movementMagnitude: Float) {
    when (currentPhase) {
        GesturePhase.NONE -> {
            if (movementMagnitude > MOVEMENT_THRESHOLD) {
                currentPhase = GesturePhase.INITIALIZING
                stabilizationCounter = 0
            }
        }
        GesturePhase.INITIALIZING -> {
            if (movementMagnitude > MOVEMENT_THRESHOLD && totalFramesInGesture > 3) {
                currentPhase = GesturePhase.MOVEMENT
            } else if (movementMagnitude < STABILIZATION_THRESHOLD) {
                stabilizationCounter++
                if (stabilizationCounter > 3) {
                    resetState()
                }
            }
        }
        GesturePhase.MOVEMENT -> {
            if (movementMagnitude <= STABILIZATION_THRESHOLD) {
                stabilizationCounter++
                if (stabilizationCounter >= 2) {
                    currentPhase = GesturePhase.STABILIZING
                    stabilizationCounter = 0
                }
            } else {
                stabilizationCounter = 0
            }
        }
        GesturePhase.STABILIZING -> {
            if (movementMagnitude > MOVEMENT_THRESHOLD) {
                currentPhase = GesturePhase.MOVEMENT
                stabilizationCounter = 0
            } else if (movementMagnitude < STABILIZATION_THRESHOLD) {
                stabilizationCounter++
                if (stabilizationCounter >= 3) {
                    currentPhase = GesturePhase.FINALIZING
                    stabilizationCounter = 0
                }
            }
        }
    }
}

```



```

GesturePhase.FINALIZING -> {
    if (movementMagnitude > MOVEMENT_THRESHOLD) {
        currentPhase = GesturePhase.MOVEMENT
        stabilizationCounter = 0
    } else {
        stabilizationCounter++
        if (stabilizationCounter >= STABILIZATION_FRAMES &&
            totalFramesInGesture >= MIN_GESTURE_FRAMES) {
            currentPhase = GesturePhase.COMPLETED
        }
    }
}

GesturePhase.COMPLETED -> {
    // Обработано в processFrame()
}

}

/**
 * Финализирует жест
 */
private fun finalizeGesture(): Pair<Boolean, List<List<Float>>> {
    if (movementHistoryBuffer.size < MIN_GESTURE_FRAMES) {
        return Pair(false, emptyList())
    }
    val selectedFrames = selectFrames()
    return Pair(true, selectedFrames)
}

/**
 * Простой отбор кадров - равномерное распределение
 */
private fun selectFrames(): List<List<Float>> {
    val bufferSize = movementHistoryBuffer.size
    if (bufferSize <= TARGET_FRAMES) {
        return movementHistoryBuffer.map { it.features }
    }
    val result = mutableListOf<List<Float>>()
    val step = (bufferSize - 1).toFloat() / (TARGET_FRAMES - 1)
    for (i in 0 until TARGET_FRAMES) {
        val index = (i * step).toInt().coerceIn(0, bufferSize - 1)
        result.add(movementHistoryBuffer[index].features)
    }
    return result
}

/**
 * Проверяет, пустой ли кадр
 */
private fun isEmptyFrame(features: List<Float>): Boolean {
    val nonZeroCount = features.count { abs(it) > 0.005f }
    return nonZeroCount < features.size * 0.1
}

/**
 * Вычисляет движение между кадрами
 */
private fun calculateMovement(prevFrame: List<Float>, currentFrame: List<Float>): Float {
    if (prevFrame.size != currentFrame.size) return 0f
    var totalMovement = 0f
    var nonZeroCount = 0
    for (i in prevFrame.indices) {

```

```

        val diff = abs(currentFrame[i] - prevFrame[i])
        if (prevFrame[i] != 0f || currentFrame[i] != 0f) {
            totalMovement += diff
            nonZeroCount++
        }
    }
    return if (nonZeroCount > 0) totalMovement / nonZeroCount else 0f
}
/**
 * Сбрасывает состояние
 */
private fun resetState() {
    currentPhase = GesturePhase.NONE
    stabilizationCounter = 0
    totalFramesInGesture = 0
    if (movementHistoryBuffer.size > 3) {
        val lastFrames = movementHistoryBuffer.takeLast(3)
        movementHistoryBuffer.clear()
        movementHistoryBuffer.addAll(lastFrames)
    }
}
}

```

Приложение 5. OverlayView

Компонент OverlayView обеспечивает визуальную обратную связь, отображая ключевые точки рук поверх видеопотока камеры. Наследуясь от Android View, он переопределяет метод draw для кастомной отрисовки. Основной вызов — корректное масштабирование и позиционирование точек с учетом пропорций экрана, что достигается с помощью метода calculateScaleFactor.

Каждая ключевая точка визуализируется как цветная окружность, а соединения между ними, определенные в HandLandmarker.HAND_CONNECTIONS, формируют "скелет" руки. Обновление результатов происходит через метод setResults, который вызывает invalidate() для синхронной перерисовки с обработкой кадров, обеспечивая плавное отображение движений в реальном времени.

Листинг 5. Код класса GesturePhaseDetector

```
/**
 * Вид для отображения ключевых точек рук поверх изображения с камеры.
 */
class OverlayView(context: Context?, attrs: AttributeSet?) :
    View(context, attrs) {
    private var results: HandLandmarkerResult? = null
    private var linePaint = Paint()
    private var pointPaint = Paint()
    private var scaleFactor: Float = 1f
    private var imageWidth: Int = 1
    private var imageHeight: Int = 1
    private var isFrontCamera: Boolean = false
    private val LANDMARK_STROKE_WIDTH = 12F
    private val LANDMARK_POINT_SIZE = 16F
    init {
        initPaints()
        Log.d(TAG, "OverlayView инициализирован")
    }
    /**
     * Очищает результаты и сбрасывает настройки отображения
     */
    fun clear() {
        results = null
        linePaint.reset()
        pointPaint.reset()
        invalidate()
        initPaints()
    }
    /**
     * Инициализирует кисти для рисования точек и линий
     */
}
```

```

private fun initPaints() {
    linePaint.color = Color.MAGENTA
    linePaint.strokeWidth = LANDMARK_STROKE_WIDTH
    linePaint.style = Paint.Style.STROKE
    pointPaint.color = Color.CYAN
    pointPaint.strokeWidth = LANDMARK_STROKE_WIDTH
    pointPaint.style = Paint.Style.FILL
}
/**
 * Устанавливает результаты определения ключевых точек
 */
fun setResults(
    handLandmarkResults: HandLandmarkerResult,
    imageHeight: Int,
    imageWidth: Int,
    isFrontCamera: Boolean = false
) {
    results = handLandmarkResults
    this.imageHeight = imageHeight
    this.imageWidth = imageWidth
    this.isFrontCamera = isFrontCamera
    calculateScaleFactor()
    invalidate()
}
/**
 * Вычисляет коэффициент масштабирования для правильного отображения ключевых точек в
 * соответствии с размерами вида и изображения
 */
private fun calculateScaleFactor() {
    if (width <= 0 || height <= 0 || imageWidth <= 0 || imageHeight <= 0) {
        scaleFactor = 1f
        return
    }
    val viewAspectRatio = width.toFloat() / height
    val imageAspectRatio = imageWidth.toFloat() / imageHeight

    scaleFactor = if (viewAspectRatio > imageAspectRatio) {
        height.toFloat() / imageHeight
    } else {
        width.toFloat() / imageWidth
    }
    scaleFactor *= 1.5f
}
/**
 * При изменении размеров вида
 */
override fun onSizeChanged(w: Int, h: Int, oldw: Int, oldh: Int) {
    super.onSizeChanged(w, h, oldw, oldh)
    calculateScaleFactor()
}
/**
 * Рисует ключевые точки рук и соединения между ними
 */
override fun draw(canvas: Canvas) {
    super.draw(canvas)
    val results = this.results ?: return
    if (results.landmarks().isEmpty()) {
        return
    }

```

```

    }
    val offsetX = (width - imageWidth * scaleFactor) / 2
    val offsetY = (height - imageHeight * scaleFactor) / 2

    for (landmark in results.landmarks()) {
        for (normalizedLandmark in landmark) {
            val x = normalizedLandmark.x()

            val pointX = x * imageWidth * scaleFactor + offsetX
            val pointY = normalizedLandmark.y() * imageHeight * scaleFactor + offsetY
            canvas.drawCircle(
                pointX,
                pointY,
                LANDMARK_POINT_SIZE, // размер точки
                pointPaint
            )
        }
    }
    HandLandmarker.HAND_CONNECTIONS.forEach {
        val startX = landmark[it!!.start()].x()
        val endX = landmark[it.end()].x()
        canvas.drawLine(
            startX * imageWidth * scaleFactor + offsetX,
            landmark[it.start()].y() * imageHeight * scaleFactor + offsetY,
            endX * imageWidth * scaleFactor + offsetX,
            landmark[it.end()].y() * imageHeight * scaleFactor + offsetY,
            linePaint
        )
    }
}
}
companion object {
    private const val TAG = "OverlayView"
}
}

```

Приложение 6. SignLanguageApiService

Класс `SignLanguageApiService` отвечает за обмен данными с сервером, используя синхронную модель взаимодействия. При завершении жеста `GesturePhaseDetector` вызывает метод `recognizeGesture`, который отправляет последовательность признаков на сервер через HTTP POST-запрос с JSON-телом и немедленно получает результат распознавания в том же запросе.

Результаты содержат название жеста, уровень уверенности и идентификатор класса, возвращаемые в едином JSON-ответе. Обработка возможных ошибок, таких как отсутствие соединения или низкий уровень уверенности, предотвращает отображение некорректных данных.

Листинг 6. Код класса `SignLanguageApiService`

```
/**
 * API-сервис для связи с сервером
 */
class SignLanguageApiService(
    private var serverUrl: String
) {
    private var lastSendTime = 0L
    private val MIN_SEND_INTERVAL = 50L
    private val TAG = "SignLanguageApiService"

    private val client = OkHttpClient.Builder()
        .connectTimeout(3, TimeUnit.SECONDS)
        .readTimeout(3, TimeUnit.SECONDS)
        .writeTimeout(3, TimeUnit.SECONDS)
        .connectionPool(ConnectionPool(5, 30, TimeUnit.SECONDS))
        .retryOnConnectionFailure(true)
        .build()

    /**
     * Обновление URL сервера
     */
    fun updateServerUrl(newUrl: String) {
        this.serverUrl = newUrl
    }

    /**
     * Отправляет данные признаков на сервер
     */
    suspend fun sendFeatures(features: List<Float>, maxRetries: Int = 3): Boolean = withContext(Dispatchers.IO) {
        val currentTime = System.currentTimeMillis()
        if (currentTime - lastSendTime < MIN_SEND_INTERVAL) {
            return@withContext true
        }
        val validSizes = listOf(126, 2520)
```

```

if (!validSizes.contains(features.size)) {
    return@withContext false
}
val nonZeroValues = features.count { it != 0f }
if (nonZeroValues < features.size * 0.1) {
    return@withContext false
}
val featuresToSend = features
if (features.size == 2520) {
    val frameSize = 126
    val numFrames = features.size / frameSize
}
var retries = 0
var lastError: Exception? = null

while (retries < maxRetries) {
    try {
        val jsonObject = JSONObject()
        val jsonArray = JSONArray()
        for (feature in featuresToSend) {
            jsonArray.put(feature)
        }
        jsonObject.put("features", jsonArray)
        jsonObject.put("timestamp", currentTime.toString())
        val requestBodyJson = jsonObject.toString()
        val requestBody = requestBodyJson.toRequestBody("application/json".toMediaType())
        val request = Request.Builder()
            .url("$serverUrl/features")
            .post(requestBody)
            .build()
        client.newCall(request).execute().use { response ->
            if (response.isSuccessful) {
                lastSendTime = currentTime
                return@withContext true
            } else if (response.code == 503) {
                delay(300)
                retries++
            } else {
                val errorBody = response.body?.string() ?: "Нет тела ответа"
                lastError = Exception("Ошибка сервера: ${response.code}")
                retries++
                delay(300)
            }
        }
    }
} catch (e: ConnectException) {
    lastError = e
    retries++
    if (retries >= maxRetries) {
        return@withContext false
    }
    delay(300)
} catch (e: SocketTimeoutException) {
    lastError = e
    retries++
    if (retries >= maxRetries) {
        return@withContext false
    }
    delay(300)
}

```

```

    } catch (e: Exception) {
        e.printStackTrace()
        lastError = e
        retries++
        if (retries >= maxRetries) {
            return@withContext false
        }
        delay(300)
    }
}
if (lastError != null) {
    //
}
return@withContext false
}
/**
 * Получает результат распознавания с сервера
 */
suspend fun getTranslation(): TranslationResponse = withContext(Dispatchers.IO) {
    try {
        val request = Request.Builder()
            .url("$serverUrl/translation")
            .get()
            .build()
        client.newCall(request).execute().use { response ->
            if (response.isSuccessful) {
                val responseBody = response.body?.string() ?: "{}"
                Log.d(TAG, "Распознан ответ: $responseBody")
                try {
                    val jsonObject = JSONObject(responseBody)
                    val gesture = jsonObject.optString("gesture", "")
                    val confidence = jsonObject.optDouble("confidence", 0.0).toFloat()
                    val classId = jsonObject.optInt("class_id", -1)
                    val timestamp = jsonObject.optLong("server_timestamp_ms", System.currentTimeMillis())

                    if (gesture.isNotEmpty()) {
                        Log.d(TAG, "Обнаружен жест: '$gesture' с уверенностью $confidence")
                    }
                    return@withContext TranslationResponse(gesture, confidence, classId, timestamp)
                } catch (e: Exception) {
                    return@withContext TranslationResponse("", 0.0f, -1, System.currentTimeMillis())
                }
            } else {
                return@withContext TranslationResponse("", 0.0f, -1, System.currentTimeMillis())
            }
        }
    } catch (e: ConnectException) {
        return@withContext TranslationResponse("", 0.0f, -1, System.currentTimeMillis())
    } catch (e: SocketTimeoutException) {
        return@withContext TranslationResponse("", 0.0f, -1, System.currentTimeMillis())
    } catch (e: Exception) {
        e.printStackTrace()
        return@withContext TranslationResponse("", 0.0f, -1, System.currentTimeMillis())
    }
}
/**
 * Проверка доступности сервера
 */

```



```

suspend fun isServerAvailable(): Boolean = withContext(Dispatchers.IO) {
    try {
        val request = Request.Builder()
            .url(serverUrl)
            .get()
            .build()
        client.newCall(request).execute().use { response ->
            val success = response.isSuccessful
            Log.d(TAG, "Проверка доступности сервера: $success")
            return@withContext success
        }
    } catch (e: ConnectException) {
        return@withContext false
    } catch (e: SocketTimeoutException) {
        return@withContext false
    } catch (e: Exception) {
        e.printStackTrace()
        return@withContext false
    }
}
}
// Класс данных для ответа перевода
data class TranslationResponse(
    val gesture: String,
    val confidence: Float,
    val class_id: Int,
    val server_timestamp_ms: Long
)

```

Приложение 7. process_gesture

Функция `process_gesture` обрабатывает HTTP POST-запросы на эндпоинт `/features`, принимая массив признаков жеста в формате JSON и немедленно возвращая результат распознавания. Она проверяет заголовок `Content-Type`, парсит данные, выполняет многоуровневую валидацию и анализирует качество признаков. Если доля ненулевых значений в массиве ниже 10%, данные отбраковываются. После успешной проверки признаки преобразуются в массивы NumPy (`float32`), передаются в модель машинного обучения, и результат распознавания возвращается клиенту в том же HTTP-ответе.

Листинг 7. Код функции `process_gesture`

```
@gesture_bp.route('/features', methods=['POST'])
def process_gesture():
    """Получает данные и сразу возвращает результат - все в одном запросе"""

    if not request.is_json:
        return jsonify({"gesture": "", "confidence": 0.0, "class_id": -1, "error": "Not JSON"}), 400

    if not AUTO_RECOGNITION_ENABLED:
        return jsonify({"gesture": "", "confidence": 0.0, "class_id": -1, "message": "Recognition disabled"})

    try:
        data = request.get_json()
        features = data.get('features', [])

        # Поддерживаем только готовые последовательности
        if len(features) not in [1260, 2520]:
            return jsonify({"gesture": "", "confidence": 0.0, "class_id": -1, "error": "Invalid features count"})

        # Проверяем модель
        model = get_model()
        if model is None:
            if not load_model():
                return jsonify({"gesture": "", "confidence": 0.0, "class_id": -1, "error": "Model not available"})
            model = get_model()

        # Обрабатываем данные
        features_np = np.array(features, dtype=np.float32)
        num_frames = len(features) // 126
        sequence = features_np.reshape((num_frames, 126))

        # Проверка качества
        if np.count_nonzero(sequence) < 50:
            return jsonify({"gesture": "", "confidence": 0.0, "class_id": -1, "message": "Low quality data"})

        # Распознаем жест
        result = recognize_gesture([frame for frame in sequence])
```

```
# Возвращаем результат сразу
if result and result.get("gesture"):
    logger.info(f"Распознан: {result['gesture']} ({result['confidence']:.3f})")
    return jsonify({
        "gesture": result["gesture"],
        "confidence": result["confidence"],
        "class_id": result["class_id"],
        "server_timestamp_ms": int(time.time() * 1000)
    })
else:
    return jsonify({"gesture": "", "confidence": 0.0, "class_id": -1, "message": "No gesture detected"})

except Exception as e:
    logger.exception(f"Ошибка обработки жеста: {e}")
    return jsonify({"gesture": "", "confidence": 0.0, "class_id": -1, "error": str(e)})
```

Приложение 8. recognize_gesture

Функция `recognize_gesture` подготавливает данные для модели машинного обучения, проверяя структуру последовательности (20 кадров по 126 признаков), тип данных (NumPy массивы) и качество через долю ненулевых элементов (порог — 3%). После успешной проверки данные нормализуются и преобразуются в формат, ожидаемый нейронной сетью. Несоответствия приводят к возврату ошибки.

Листинг 8. Код функции `recognize_gesture`

```
def recognize_gesture(features_sequence):
    """Распознавание жестов из последовательности."""
    model = get_model()

    if not AUTO_RECOGNITION_ENABLED:
        return {"gesture": "", "confidence": 0.0, "class_id": -1}
    if model is None:
        return {"gesture": "Error: Model not loaded", "confidence": 0.0, "class_id": -1}

    try:
        if not isinstance(features_sequence, (list, tuple)) or not all(isinstance(item, np.ndarray) for item in features_sequence):
            return {"gesture": "Error: Invalid input data type", "confidence": 0.0, "class_id": -1}

        if len(features_sequence) != 20:
            return {"gesture": "Error: Invalid sequence length", "confidence": 0.0, "class_id": -1}

        for i, frame in enumerate(features_sequence):
            if frame.shape != (126,):
                return {"gesture": "Error: Invalid frame shape", "confidence": 0.0, "class_id": -1}

        input_data_np = np.array(features_sequence, dtype=np.float32)

        # Проверка качества данных
        non_zero_count = np.count_nonzero(input_data_np)
        total_elements = input_data_np.size
        non_zero_percentage = (non_zero_count / total_elements) * 100 if total_elements > 0 else 0

        if non_zero_percentage < 3.0:
            return {"gesture": "", "confidence": 0.0, "class_id": -1}
        input_data = np.expand_dims(input_data_np, axis=0)

        start_time = time.time()
        predictions = model.predict(input_data, verbose=0)
        prediction_time = time.time() - start_time

        scaled_predictions = predictions[0]
        predicted_class_index = np.argmax(scaled_predictions)
        confidence = float(scaled_predictions[predicted_class_index])

        confidence_threshold = Config.CONFIDENCE_THRESHOLD
```

```

if confidence >= confidence_threshold:
    gesture_name = CLASS_MAPPING.get(predicted_class_index, f"Unknown_ID_{predicted_class_index}")
    class_id_to_return = int(predicted_class_index)
    try:
        top_5_indices = np.argsort(predictions[0])[-5:][::-1]
        top_5_values = predictions[0][top_5_indices]
        top_5_labels = [CLASS_MAPPING.get(idx, f"Unknown_{idx}") for idx in top_5_indices]
        top_5_info = [f"{label}: {value:.3f}" for label, value in zip(top_5_labels, top_5_values)]
        logger.info(f"Жест распознан: {gesture_name} (id: {class_id_to_return}) | Уверенность: {confidence:.3f} | Top-5: {' '.join(top_5_info)}")
    except Exception as e_top5:
        return {
            "gesture": gesture_name,
            "confidence": confidence,
            "class_id": class_id_to_return
        }
    else:
        gesture_name = ""
        confidence_to_return = 0.0
        class_id_to_return = -1

    top_class_name = CLASS_MAPPING.get(predicted_class_index, f"Unknown_ID_{predicted_class_index}")
    logger.info(f"Низкая уверенность ({confidence:.3f} < {confidence_threshold:.3f}). Наиболее вероятный класс: {top_class_name} (id: {predicted_class_index}), but result not returned.")
    return {
        "gesture": gesture_name,
        "confidence": confidence_to_return,
        "class_id": class_id_to_return
    }
except Exception as e:
    return {
        "gesture": "Recognition error",
        "confidence": 0.0,
        "class_id": -1
    }

```